

**VIETNAM NATIONAL UNIVERSITY
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



BACHELOR'S THESIS

Major: Computer Science

**OPTIMIZING GAME BALANCE IN ROGUELIKE
DECKBUILDERS VIA MULTI-OBJECTIVE
EVOLUTIONARY ALGORITHMS AND
REINFORCEMENT LEARNING**

Supervisor: Assoc. Prof. Dr. Nguyễn Trí Thành
Student: Nguyễn Nhật Phong
Student ID: 22028272
Class: QH-2022-I/CQ-I-CS2

HA NOI - 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Khoa học máy tính

**TỐI ƯU HÓA CÂN BẰNG TRONG TRÒ
CHƠI ROGUELIKE DECKBUILDER
BẰNG THUẬT TOÁN TIẾN HÓA ĐA
MỤC TIÊU VÀ HỌC TĂNG CƯỜNG**

Cán bộ hướng dẫn: PGS. TS. Nguyễn Trí Thành

Sinh viên thực hiện: Nguyễn Nhật Phong

Mã sinh viên: 22028272

Lớp: QH-2022-I/CQ-I-CS2

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Nguyễn Nhật Phong

**OPTIMIZING GAME BALANCE IN
ROGUELIKE DECKBUILDERS VIA
MULTI-OBJECTIVE EVOLUTIONARY
ALGORITHMS AND REINFORCEMENT
LEARNING**

BACHELOR'S THESIS

Major: Computer Science

Supervisor: Assoc. Prof. Dr. Nguyễn Trí Thành

HANOI - 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nguyễn Nhật Phong

**TỐI ƯU HÓA CÂN BẰNG TRONG TRÒ
CHƠI ROGUELIKE DECKBUILDER
BẰNG THUẬT TOÁN TIẾN HÓA ĐA
MỤC TIÊU VÀ HỌC TĂNG CƯỜNG**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Khoa học máy tính

Cán bộ hướng dẫn: PGS. TS. Nguyễn Trí Thành

HÀ NỘI - 2026

TÓM TẮT

Tóm tắt: Cân bằng trò chơi là thách thức lớn trong phát triển game, đặc biệt với Roguelike Deckbuilder nơi hàng trăm tham số - sát thương thẻ bài, chỉ số kẻ thù, hiệu ứng cổ vật và phân bố bản đồ - phải tạo trải nghiệm công bằng và hấp dẫn. Tinh chỉnh thủ công tốn thời gian, dễ sai sót và thiếu tính hệ thống.

Luận văn trình bày hệ thống tự động cân bằng game cho prototype Roguelike Deckbuilder xây dựng bằng Python 3.12, gồm ba tầng tích hợp. Tầng Game Core hiện thực logic trò chơi: chiến đấu theo lượt, bản đồ 15 tầng sinh ngẫu nhiên, bộ bài hơn 30 thẻ và 20 cổ vật, cùng nhiều loại phòng. Tác nhân heuristic tắt định (DHA) xác nhận tính chơi được và thiết lập chỉ số cơ sở.

Đóng góp cốt lõi là bốn tác nhân Học Tăng Cường huấn luyện bằng MaskablePPO với lối chơi riêng biệt - Tấn công, Phòng thủ, Cân bằng và Thích nghi - định hình qua hàm phần thưởng tùy chỉnh và huấn luyện 14 triệu bước trên GPU. Che mặt nạ hành động và ngẫu nhiên hóa miền nâng cao ổn định và khả năng tổng quát. Bốn tác nhân đạt tỷ lệ thắng 24 - 34%.

Tầng Tối ưu hóa triển khai hai phương pháp: Thuật toán Di truyền đơn mục tiêu trên ~500 tham số và NSGA-II với bộ gen phân cấp 38 tham số tối ưu đồng thời ba mục tiêu - Cân bằng, Thu hút và Tính nhất quán. Qua 30 thế hệ, quần thể 50, NSGA-II đạt Cân bằng 0,960, Thu hút 0,900 và Tính nhất quán 0,984, loại bỏ thẻ bẫy, tăng thẻ có giá trị từ 12 lên 14 trong 49 phút - nhanh hơn 33% so với GA đơn mục tiêu.

Nghiên cứu cho thấy kết hợp bộ gen phân cấp, tối ưu đa mục tiêu và tác nhân RL tạo framework cân bằng tự động hiệu quả, vượt trội so với tinh chỉnh thủ công và di truyền đơn mục tiêu.

Từ khóa: *Cân bằng trò chơi, Roguelike Deckbuilder, NSGA-II, Học tăng cường.*

ABSTRACT

Abstract: Game balancing is one of the most critical and labor-intensive challenges in game development, particularly for Roguelike Deckbuilder games, where hundreds of interconnected parameters - card damages, enemy statistics, relic effects, and map distributions - must coexist to produce a fair and engaging player experience. Traditional manual tuning is slow, error-prone, and incapable of systematically exploring the vast parameter space.

This thesis presents a complete automated game-balancing framework for a Roguelike Deckbuilder prototype built in Python 3.12, composed of three tightly integrated layers. The Core Game Layer implements full game logic: turn-based combat, a procedurally generated 15-floor map, a deckbuilding system with over 30 cards and 20 relics, and multiple room types. A rule-based Deterministic Heuristic Agent (DHA) was developed first to validate playability and establish baseline metrics.

The central contribution is a suite of four Reinforcement Learning (RL) agents trained via Maskable Proximal Policy Optimization (MaskablePPO), each embodying a distinct play style - Aggressive, Defensive, Balanced, and Adaptive - shaped through custom reward functions and trained over 14 million timesteps on GPU hardware. Invalid Action Masking and Domain Randomization are employed to improve training stability and agent generalization across varied game configurations. All four agents achieve win rates of 24 - 34%, a realistic benchmark given the game's inherent stochastic elements.

The Optimization Layer employs two evolutionary approaches: a Single-Objective Genetic Algorithm (GA) operating on ~ 500 direct parameters, and NSGA-II with a hierarchical genome of 38 parameters that optimizes three objectives simultaneously - Balance, Engagement, and Coherence. After 30 generations with a population of 50, NSGA-II achieves a Balance Score of 0.960, Engagement Score of 0.900, and Coherence of 0.984, eliminates all trap cards, and raises the number of viable cards from 12 to 14 - all in 49 minutes, 33% faster than the single-objective GA.

This work demonstrates that combining hierarchical genome encoding, multi-objective optimization, and learned RL agents produces a robust and extensible automatic balancing framework that significantly outperforms both manual tuning and flat single-objective evolutionary approaches.

Keywords: *Roguelike Deckbuilder, NSGA-II, Reinforcement Learning, Game Balance Optimization.*

DECLARATION OF AUTHORSHIP

I hereby declare that this graduation thesis is my own original work. All results, analyses, and conclusions presented in this document are truthful and have not been copied from any other source in any form whatsoever. All references to external sources have been properly cited and acknowledged in accordance with academic regulations.

If any form of misconduct is discovered, I accept full responsibility for the content of this thesis. The University of Engineering and Technology bears no liability for any copyright or intellectual property violations I may have committed during the preparation of this work.

Hanoi, May 2026

Author

Nguyễn Nhật Phong

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, Assoc. Prof. Dr. Nguyễn Trí Thành, for his invaluable guidance, insightful feedback, and continuous encouragement throughout the entire process of completing this thesis. His expertise in algorithms and artificial intelligence has shaped both the direction and quality of this research.

I am also grateful to the faculty and staff of the Faculty of Information Technology, University of Engineering and Technology, Vietnam National University, for providing me with a solid academic foundation over four years of study. The knowledge and skills I have gained here have made this project possible.

Special thanks go to my fellow students and friends who provided moral support, engaged in stimulating technical discussions, and helped test early versions of the game system. Their enthusiasm and feedback were a constant source of motivation.

Finally, I owe immeasurable gratitude to my family, whose unwavering love, patience, and encouragement have sustained me through every challenge. This work is dedicated to them.

Hanoi, May 2026

Nguyễn Nhật Phong

Contents

Tóm tắt	i
Abstract	ii
Declaration of Authorship	iv
Acknowledgements	v
Table of Contents	vi
List of Figures	x
List of Tables	xiii
List of Abbreviations	xiv
Introduction	1
1 BACKGROUND AND RELATED WORK	8
1.1 Roguelike Deckbuilder Games	8
1.1.1 Roguelike Deckbuilders	8
1.1.2 Roguelike Games	9
1.1.3 Deckbuilding Mechanics	9
1.1.4 Balancing Challenges in Roguelike Deckbuilders	9
1.2 Reinforcement Learning	10
1.2.1 Why RL for Sub-Problem 2: Intelligent Game Evaluation	10
1.2.2 Markov Decision Processes	11
1.2.3 Mapping the MDP to Our Game	12
1.2.4 Deep Reinforcement Learning	13
1.2.5 Proximal Policy Optimization (PPO)	13
1.2.6 Invalid Action Masking	14
1.2.7 Domain Randomization	14

1.3	Optimization Algorithms	15
1.3.1	Survey of Automated Game Balancing Approaches	15
1.3.2	Genetic Algorithms	16
1.3.3	NSGA-II: Multi-Objective Evolutionary Optimization	17
1.4	Related Work	20
1.4.1	Automated Game Balancing	20
1.4.2	RL for Card Games	20
1.4.3	Procedural Content Generation	20
2	SYSTEM ANALYSIS AND DESIGN	21
2.1	Requirements Analysis	21
2.1.1	Functional Requirements	21
2.1.2	Non-Functional Requirements	22
2.2	System Architecture	22
2.3	Core Game Engine Design	24
2.3.1	Game Structure Overview	24
2.3.2	Combat System Design	25
2.4	Deterministic Heuristic Agent Design	26
2.4.1	Purpose and Role	26
2.4.2	Decision Logic	26
2.5	Reinforcement Learning Environment Design	27
2.5.1	Observation Space	27
2.5.2	Action Space	27
2.5.3	Episode Structure	29
2.6	RL Agent Persona Design	29
2.6.1	Design Rationale	29
2.6.2	Aggressive Agent	29
2.6.3	Defensive Agent	30
2.6.4	Balanced Agent	30
2.6.5	Adaptive Agent	31
2.7	Optimization Layer Design	32
2.7.1	Data Flow	32
2.7.2	Genome Representations	32
2.7.3	Fitness Functions	33
3	IMPLEMENTATION	35
3.1	Development Environment	35

3.2	Core Game Engine Implementation	36
3.2.1	Project Structure	36
3.2.2	Game Controller	36
3.2.3	Combat System	36
3.2.4	Map Generation	37
3.2.5	Data-Driven Design	37
3.2.6	Heuristic AI Implementation	37
3.3	Reinforcement Learning Environment	37
3.3.1	RoguelikeEnv (Gymnasium Wrapper)	37
3.3.2	Observation Space	38
3.3.3	Action Space	38
3.3.4	Domain Randomization	39
3.4	Agent Training Pipeline	39
3.4.1	Three-Phase Training Strategy	39
3.4.2	MaskablePPO Hyperparameters	39
3.4.3	Deck Quality Reward	40
3.4.4	Training Infrastructure	40
3.5	Evolutionary Optimization Implementation	41
3.5.1	Genome Representation	41
3.5.2	Simulation Runner	41
3.5.3	Fitness Evaluator	41
3.5.4	NSGA-II via pymoo	42
4	EXPERIMENTS AND EVALUATION	43
4.1	Overview of Evaluation	43
4.1.1	Why These Metrics?	43
4.2	Experimental Setup	44
4.2.1	Hardware Configuration	44
4.2.2	Optimization Algorithm Parameters	45
4.2.3	Target Metrics	45
4.3	RL Agent Training Results	45
4.3.1	Final Win Rates	46
4.3.2	Behavioral Differentiation	47
4.3.3	Training Convergence	48
4.4	Game Parameter Optimization Results	49
4.4.1	Single-Objective GA Results	50
4.4.2	NSGA-II Multi-Objective Results	51

4.4.3	Pareto Front Analysis	53
4.4.4	Comparative Summary	55
4.5	Card Viability Analysis	56
4.6	Comparison with Heuristic Agent Baseline	57
4.7	Discussion	58
4.7.1	Why NSGA-II Outperforms GA	58
4.7.2	Limitations of Current Evaluation	58
4.7.3	Comparison with Related Automated Balancing Work	59
	CONCLUSION	60
	APPENDIX	63
	References	78

List of Figures

1.1	Overview of the Roguelike Deckbuilder prototype used in this thesis. A run proceeds through three Acts (15 floors total). Each floor presents one of five room types (legend, bottom left); Monster and Elite rooms trigger the turn-based combat cycle (bottom right), which awards a new card upon victory. The run ends in Victory on defeating the Floor-15 Boss, or Death when HP reaches zero.	10
2.1	High-level system architecture showing data flow between the Optimization Layer, Simulation/Training Layer, Data Layer, and Core Game Layer.	23
2.2	Class diagram of the core game module showing the relationships between <i>GameController</i> , <i>CombatManager</i> , <i>DeckManager</i> , the AI interface, and the optimization layer.	25
2.3	Sequence diagram of one combat turn: the Combat Manager draws cards from the Deck Manager, queries the AI Agent for a decision, resolves the card action, executes the enemy turn, and checks win/loss conditions.	26
2.4	Data flow in the optimization loop. A genome is created by the optimization layer, applied to a scaled <i>GameData</i> copy, passed to the simulation runner which uses the AI agent to play games, and resulting statistics are evaluated into fitness scores that feed back to the optimizer.	32
4.1	Performance comparison of the four PPO agents across win rate, average HP remaining on victory, and average floor reached on death. Evaluated over 200 deterministic episodes. The Adaptive agent achieves the highest win rate (34.0%) while maintaining a competitive HP on win (29.8%), demonstrating the effectiveness of emergent strategy switching.	47
4.2	Radar chart comparing the four RL agents across five normalised dimensions: win rate, HP on victory, turns efficiency (inverse of average turns per win), floor depth on death, and cross-episode consistency. Each agent occupies a distinct region, confirming that reward shaping successfully produced differentiated playing styles.	48

4.3	Win-rate training curves for all four PPO agents over 14 million steps. Two vertical dashed lines mark the phase boundaries: Phase 1 (easy maps) to Phase 2 (full maps) at 5M steps, and Phase 2 to Phase 3 (domain randomisation) at 10M steps. A brief win-rate dip is visible at each boundary - first when the easy-win floor is removed, then when DR noise is introduced - followed by recovery as training continues. The Aggressive agent's flat plateau in Phase 3 reflects early policy saturation.	49
4.4	Box plots of normalised episode return distributions over 200 evaluation episodes per agent. The Defensive agent shows the highest median return but a narrower spread, reflecting its conservative strategy. The Aggressive agent exhibits the widest interquartile range, consistent with high-risk offensive play that either terminates quickly on a win or dies early.	49
4.5	Weighted fitness of the best individual across 30 GA generations ($0.5 \times \text{Balance} + 0.3 \times \text{Engagement} + 0.2 \times \text{Coherence}$). Fitness improves from 0.745 to 0.841 (+12.9%) as the single-objective GA progressively tunes card and enemy parameters toward the target balance metrics.	50
4.6	Evolution of Balance, Engagement, and Coherence scores for the best GA individual across 30 generations. Balance improves from 0.754 to 0.884 (+17.2%); Engagement from 0.655 to 0.724 (+10.5%); Coherence from 0.853 to 0.910 (+6.7%).	51
4.7	NSGA-II hypervolume indicator across 30 generations. The hypervolume (approximated as the product of the three best-individual objective scores) rises from 0.084 in generation 1 to 0.874 in generation 30 (+941%), reflecting simultaneous improvement across all three objectives.	52
4.8	Best Balance, Engagement, and Coherence scores across 30 NSGA-II generations for the best individual each generation. The S-curve shape is characteristic of evolutionary optimization: rapid initial gains followed by diminishing returns as the algorithm refines an already-good solution.	52
4.9	Three-dimensional Pareto front in (Balance, Engagement, Coherence) space. Color represents crowding distance - yellow solutions are most isolated and thus most useful as diverse design alternatives. Thirty non-dominated solutions were identified across the 30-generation run.	54
4.10	Two-dimensional projections of the NSGA-II Pareto front. Each point represents one non-dominated solution.	55

4.11	Card pick rate versus win rate when picked, colored by card type (blue: Attack, teal: Skill/Power). Ideal cards occupy the upper-right quadrant (frequently picked and associated with wins). Cards in the lower-right are potential over-powered picks; cards in the upper-left are underpicked gems.	56
4.12	Viable card count (green, top line) and trap card count (pink, bottom line) across 30 NSGA-II generations. The viable card count grows from 10 to 14; trap cards are eliminated entirely by generation 22 as coherence converges toward 1. . . .	57

List of Tables

2.1	Observation space components (total: 152 dimensions).	28
3.1	Development environment specifications.	35
3.2	Observation space components (total: 152 dimensions).	38
3.3	MaskablePPO hyperparameters for all four agents across the three training phases.	40
3.4	HierarchicalGenome parameter groups (38 continuous dimensions).	41
4.1	Hardware configuration used in experiments.	44
4.2	Comparison of GA and NSGA-II algorithm configurations.	45
4.3	Final win rates of the four RL agents evaluated over 200 deterministic episodes.	46
4.4	Comprehensive comparison of GA and NSGA-II performance after 30 generations.	55
4.5	Comparison of optimization outcomes using the Deterministic Heuristic Agent (DHA) versus RL agents as evaluators in NSGA-II (30 generations, 100 individuals).	58
A.1	152-dimensional observation vector layout. All values are normalized to $[0, 1]$ unless noted.	67
A.2	Complete reward function summary for all four RL agents.	69

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CLR	Common Language Runtime
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DLL	Dynamic Link Library
DQN	Deep Q-Network
DR	Domain Randomization
DRL	Deep Reinforcement Learning
GA	Genetic Algorithm
GPU	Graphics Processing Unit
HP	Hit Points (Health Points)
JSON	JavaScript Object Notation
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
MOEA	Multi-Objective Evolutionary Algorithm
NSGA-II	Non-dominated Sorting Genetic Algorithm II
PCG	Procedural Content Generation
PPO	Proximal Policy Optimization
RAM	Random Access Memory
RL	Reinforcement Learning
RNG	Random Number Generator
SBX	Simulated Binary Crossover
SDK	Software Development Kit
SB3	Stable-Baselines3
UET	University of Engineering and Technology
VNU	Vietnam National University

INTRODUCTION

This chapter provides an overview of the thesis: it motivates the research problem, introduces the game prototype with a concrete balance example, formally defines the optimization objectives, states each research objective mapped to its corresponding chapter, and summarizes the main contributions.

1. Motivation

The video game industry has grown into one of the largest entertainment sectors in the world, generating over \$180 billion in annual revenue. Within this landscape, the Roguelike Deckbuilder sub-genre has seen remarkable popularity, with titles such as Slay the Spire, Monster Train, and Inscryption achieving both critical acclaim and millions of copies sold [1]. These games combine procedurally generated maps, permadeath mechanics, and an evolving card collection that the player builds across each run.

Despite their commercial success, Roguelike Deckbuilders are notoriously difficult to balance. A well-balanced game must satisfy several competing requirements: it should be challenging enough to remain engaging but not so punishing that players quit in frustration; the card pool should support multiple viable strategies so that no single best deck dominates; and the difficulty should increase smoothly across fifteen or more floors so that no single floor becomes an impassable wall. Satisfying all of these properties across the combinatorial space of possible card combinations, enemy encounters, relic synergies, and random deck draws is a task that defies purely manual tuning [2].

Traditional balancing relies on playtesting: human players attempt the game, report problems, and designers manually adjust parameters. This process is slow, expensive, and bounded by the design team’s available time. It is also inherently subjective: what feels fair to an expert player may seem impossible to a beginner. Automated simulation provides a data-driven alternative, but raises a key question: what AI agent should simulate the player?

This thesis addresses both problems together. We design and train a suite of Reinforcement Learning (RL) AI agents that learn to play the game, and then use these agents as evaluators inside an evolutionary optimization loop that searches for well-balanced game configurations.

2. The Game and the Concept of Balance

2.1 The Game Prototype

The experimental platform for this thesis is a custom Roguelike Deckbuilder game prototype implemented in Python. The game proceeds as a series of *runs*. Each run places the player character on a 15-floor map divided into three Acts (floors 1 to 5, 6 to 10, and 11 to 15). The player begins with a small starter deck and a fixed amount of HP. At each floor, the player selects one room from the available nodes on the map. Rooms come in five types: Monster (mandatory combat), Elite (a harder combat with better rewards), Rest Site (heal or permanently remove a card from the deck), Shop (buy or remove cards with gold), and Event (a text-based choice with random outcomes).

Combat is turn-based. Each turn, the player draws five cards and has three mana points to spend. Playing a card consumes mana according to its cost and triggers its effect - dealing damage to enemies, gaining block (temporary damage reduction), or applying a status effect such as Poison or Burn. After the player ends their turn, each enemy executes its scheduled action: attacking, buffing itself, or debuffing the player. Block is cleared at the end of each turn. Combat ends when all enemies are defeated (the player wins the room) or the player's HP reaches zero (the run ends in defeat).

After each Monster or Elite room, the player chooses one of three randomly offered cards to add to their deck. Rest Sites allow healing or card removal. After defeating all fifteen floors and the final boss, the run ends in victory. The game contains over 30 distinct cards organized into three types (Attack, Skill, Power) with star ratings 1 to 5, over 20 passive relics, and a range of enemy types that increase in difficulty across Acts.

2.2 What is Game Balance? A Concrete Example

Game balance means the game is neither too easy nor too hard, and that many different deck-building strategies can lead to victory. To make this concrete, consider a single parameter: the damage value of the card *Flame Strike*, a 2-mana Attack card.

Unbalanced (too easy): If *Flame Strike* deals 40 damage while most early enemies have only 15 HP, the player clears every combat in the first turn with two cards. Win rates approach 90%. The game becomes trivial and unengaging - players never need to manage block, plan turn order, or explore card synergies.

Unbalanced (too hard): If *Flame Strike* deals 3 damage while enemies deal 10 damage

per attack and have 50 HP, the player can never kill enemies fast enough to prevent fatal damage accumulation. Win rates drop below 5%. The game becomes unwinnable and frustrating.

Balanced: With Flame Strike at 8 damage, enemies at roughly 20 to 30 HP, and enemies dealing 6 to 9 damage per attack, the player must combine multiple cards and manage block carefully to survive. Win rates settle around 30 to 40% for a well-trained AI agent - victory is achievable but demands good decisions. Multiple strategies can succeed: a burn-focused deck, a block-heavy defensive build, or a mixed approach all remain viable, keeping the game engaging.

Balancing the full game means finding the right values for *all* parameters simultaneously - every card's damage, every enemy's HP and attack power, mana costs, room frequencies, healing rates, and more. Approximately 500 individual values must be tuned together. This is the core problem this thesis addresses.

3. Problem Statement

Let $\theta \in \mathbb{R}^n$ be a vector of game parameters. In our game, θ includes values such as the base damage of Flame Strike (nominally 8), the HP of Goblin enemies (nominally 25), each card's mana cost, room type distribution weights, and rest site healing rates - approximately 500 individual parameters in total. Given a game simulator $G(\theta)$ and a set of AI agents $\mathcal{A} = \{a_1, a_2, \dots, a_k\}$, we wish to find θ^* such that the resulting game satisfies:

1. **Balance:** The win rate $w(\theta, a)$ for a representative AI agent a lies within a target range $[w_{\min}, w_{\max}]$, and the game neither trivializes nor becomes unwinnable. In our experiment, the target is $w \in [35\%, 55\%]$, directly grounded in the Flame Strike example above.
2. **Engagement:** A large fraction of available cards are viable - choosing them correlates positively with winning - supporting diverse build strategies. A game where only three cards are worth picking fails this criterion.
3. **Coherence:** No card is a dominant trap - one that appears powerful but consistently leads to defeat - and all parameter values remain within an interpretable and realistic range.

Because these three objectives can conflict (making the game harder may improve balance but reduce engagement by limiting viable strategies), we frame the search as a multi-objective optimization problem over the Pareto front of (Balance, Engagement, Coherence).

This overarching problem naturally decomposes into three technically distinct sub-problems, each requiring a dedicated system component:

1. **Sub-Problem 1 - Core Game Engine:** Implement a complete, data-driven game simulation that faithfully replicates the Roguelike Deckbuilder rules and exposes all balance parameters through a modifiable interface. This is the prerequisite for both automated evaluation and optimization - without a fast, parametrically tunable simulator, neither the agents nor the optimizer can function. The game-design foundations are covered in Chapter 1, Section 1.1; the engine design and implementation appear in Chapters 2 and 3.
2. **Sub-Problem 2 - AI Agent for Game Evaluation:** Design and train AI agents that can play the game and provide a meaningful, diverse quality signal. A single deterministic agent captures only one play style and may miss balance problems visible only under a different strategy. We address this by combining a fast Deterministic Heuristic Agent (DHA) for optimization throughput with four MaskablePPO RL personas (Aggressive, Defensive, Balanced, Adaptive) for behavioral diversity and final validation. The RL theoretical background is in Chapter 1, Section 1.4; agent design and training are in Chapters 2 and 3.
3. **Sub-Problem 3 - Multi-Objective**
Game Balance Optimization: Search the ~ 500 -dimensional parameter space θ to find configurations that simultaneously satisfy all three objectives. Because the objectives conflict, a multi-objective algorithm returning a *Pareto front* of trade-off solutions is more informative than a fixed-weight scalar fitness function - it preserves designer agency by revealing the actual trade-off space rather than committing to a single weighting prior to the search. The GA and NSGA-II algorithmic background is in Chapter 1, Sections 1.2 and 1.3; the optimizer design and evaluation appear in Chapters 2, 3, and 4.

These sub-problems are interdependent: Sub-Problem 3 cannot begin until Sub-Problems 1 and 2 are solved, and the quality of the optimization result is bounded by the fidelity of both the simulation and the evaluation agents. The three sub-problems also map directly onto the four-layer system architecture presented in Chapter 2.

4. Research Objectives

The concrete objectives of this thesis, each mapped to the chapter where it is primarily addressed, are:

1. **Build a complete game prototype** (Chapters 2 and 3): Implement a fully playable Roguelike Deckbuilder in Python with turn-based combat, a procedurally generated map, a card and relic system, and a data-driven parameter system loadable from JSON.

2. **Establish a playability baseline** (Chapters 2 and 3): Design and implement a rule-based Deterministic Heuristic AI Agent (DHA) that confirms the game is playable and provides a fast initial balancing signal.
3. **Train four RL AI agent personas** (Chapters 2, 3, and 4): Replace the deterministic agent with four learned MaskablePPO AI agents representing distinct play styles (Aggressive, Defensive, Balanced, Adaptive), each trained with persona-specific reward shaping and Domain Randomization to generalize across varied game configurations.
4. **Implement and compare two evolutionary optimizers** (Chapters 2, 3, and 4): Deploy a Single-Objective Genetic Algorithm (GA) and the NSGA-II multi-objective algorithm; compare their effectiveness at discovering balanced game configurations.
5. **Analyze and visualize results** (Chapter 4): Produce Pareto front analysis, card viability charts, and training convergence curves to demonstrate that the research objectives have been met.

5. Scope and Methodology

The research covers the full software stack from game engine to optimization layer. The game prototype is designed as a research vehicle rather than a commercial product; its parameter space is representative of the real design challenge without requiring a production codebase.

The work proceeds through four phases:

1. **Foundation (Chapter 1):** Literature review on Roguelike games, evolutionary algorithms, reinforcement learning, and related automated balancing research.
2. **System design (Chapter 2):** Define requirements, system architecture, AI agent designs, and optimization formulations.
3. **Implementation (Chapter 3):** Implement the Python game engine, the Gymnasium RL environment, and the optimization modules.
4. **Evaluation (Chapter 4):** Train AI agents, run GA and NSGA-II experiments, and analyze results.

6. Main Contributions

The primary contributions of this thesis, each directly addressing one or more of the stated objectives, are:

- A complete, open-source Roguelike Deckbuilder prototype with a data-driven architecture that separates game logic from balance parameters, enabling fully automated parameter search (addresses Objective 1).
- Four trained RL AI agent personas exhibiting measurably distinct behaviors - win rates from 24% to 34% with statistically significant behavioral differences - covering the diversity of strategies needed for meaningful balance testing (addresses Objective 3).
- A hierarchical genome encoding that compresses approximately 500 raw game parameters into 38 semantically coherent continuous dimensions, enabling faster and more coherent optimization convergence (addresses Objectives 1 and 4).
- An empirical comparison of GA versus NSGA-II for game balancing under identical simulation budgets, demonstrating that the hierarchical NSGA-II converges 33% faster while achieving superior scores on all three objectives (addresses Objective 4).
- A reusable framework applicable to other card-based or turn-based games with minor adaptation.

7. Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 1 - Background and Related Work:** Surveys theory organized around the three sub-problems. Section 1.1 covers Sub-Problem 1 foundations: Roguelike Deckbuilder mechanics and automated balancing challenges. Sections 1.2 and 1.3 cover Sub-Problem 3 foundations: a comparative survey of game balancing approaches, followed by the Genetic Algorithm and NSGA-II algorithms with explicit mapping to the game objectives. Section 1.4 covers Sub-Problem 2 foundations: the motivation for intelligent evaluation agents, Reinforcement Learning theory (MDP, PPO, invalid action masking), and Domain Randomization. Section 1.5 reviews related prior work and positions this thesis relative to the literature.

- **Chapter 2 - System Analysis and Design:** Presents the requirements analysis, the four-layer architecture, the game engine design, the RL environment specification, the four AI agent persona designs, and the optimization layer design including genome representations and fitness functions.
- **Chapter 3 - Implementation:** Details the Python game engine implementation, the Gymnasium RL environment, the three-phase MaskablePPO training pipeline, and the GA/NSGA-II optimizer implementations.
- **Chapter 4 - Experiments and Evaluation:** Reports results for both the RL training phase and the evolutionary optimization phase, including comparative discussion and interpretation of how results demonstrate achievement of the research objectives.
- **Conclusion:** Summarizes the achievements relative to the stated objectives, identifies current limitations, and outlines future research directions.

CHAPTER 1

BACKGROUND AND RELATED WORK

This chapter surveys the theoretical and technical foundations for the three sub-problems identified in the Introduction, organized in a top-down manner that introduces each topic at the genre level before drilling into its components. Section 1.1 covers Sub-Problem 1 (game engine background): it opens with Roguelike Deckbuilders as a combined genre before separately examining the Roguelike and Deckbuilding mechanics that compose it, then characterizes the balancing challenges unique to the genre. Section 1.2 covers Sub-Problem 2 (AI agent background): motivated by the need for intelligent evaluation agents, it develops the Reinforcement Learning theory including MDP, PPO, invalid action masking, and Domain Randomization. Section 1.3 covers Sub-Problem 3 (optimization background): a survey of game balancing approaches leads into the Genetic Algorithm and NSGA-II, with explicit mapping of each algorithm’s parameters to the game-balancing context. Section 1.4 reviews related prior work. Each section closes by connecting the theory to the design decisions in Chapter 2.

1.1 Roguelike Deckbuilder Games

1.1.1 *Roguelike Deckbuilders*

Roguelike Deckbuilders are a hybrid genre that merges procedural dungeon-crawling with card-driven combat and progressive deck construction. In a typical run, the player traverses a branching map of rooms, fights enemies using a hand drawn from their personal deck, and acquires new cards after victories - all under the threat of permadeath that resets the run on defeat. The genre therefore demands both tactical in-combat card play and strategic long-run deck curation simultaneously. Formally, a run can be characterized as:

- Each run generates a unique map of rooms, enemies, and reward opportunities.
- After each combat, players select cards to add to their deck from a randomized reward pool.
- Death resets all progress, but accumulated strategic knowledge transfers.
- Victory requires both skillful in-combat card play and astute long-term deck construction.

Slay the Spire [1], released in 2019, is widely credited with establishing the genre’s commercial viability. The game’s deep card pool and enemy variety make it an ideal testbed for automated balancing research. This thesis builds a custom prototype inspired by Slay the Spire’s design principles.

1.1.2 Roguelike Games

The Roguelike genre traces its origins to Rogue (1980), a terminal-based dungeon crawler distinguished by two defining features [3]. First, procedural generation: the dungeon layout, enemy placement, and item distribution are randomly generated anew for each attempt, ensuring that no two runs share the same experience. Second, permadeath: when the player character dies, all progress is lost and the player must begin a fresh run from the very start. Together, these mechanics create a gameplay loop that rewards learning and mastery over brute-force repetition.

Modern Roguelikes typically add turn-based gameplay, resource management (health, mana, gold), and a meta-progression system that unlocks new content across multiple runs. The genre rewards deep strategic thinking precisely because each decision can be decisive - there are no mid-run saves to fall back upon.

1.1.3 Deckbuilding Mechanics

The deckbuilder is a game category in which players begin with a small, weak deck of cards and progressively improve it by acquiring new cards and removing poor ones. Popularised by board games such as Dominion (2008) and Thunderstone (2009), the mechanic creates a layered strategic challenge: players must evaluate short-term combat utility against long-term deck synergy, deciding which cards to add based on an evolving game state. A well-constructed deck becomes exponentially more powerful than the sum of its parts through synergies - combinations of cards whose joint effect far exceeds their individual contributions.

1.1.4 Balancing Challenges in Roguelike Deckbuilders

Balancing a Roguelike Deckbuilder is substantially harder than balancing a deterministic game for several reasons:

1. Combinatorial card interactions: With n cards in the pool, there are 2^n possible subsets that could form a player’s deck, each with different synergies.

2. Stochastic elements: Random card draws, enemy intent patterns, and reward-pool composition introduce variance that makes it difficult to isolate balance problems from luck.
3. Emergent strategies: Players frequently discover synergies that designers did not anticipate, creating dominant strategies that must be patched post-release.
4. Player skill heterogeneity: A configuration balanced for expert players may be prohibitively difficult for novices, and vice versa.

The custom game prototype used in this thesis follows the design principles described above. Figure 1.1 shows the full run structure: 15 floors divided into three Acts, the five room types encountered along the way, and the turn-based combat cycle that drives card acquisition. The winning condition is to defeat the final Boss on Floor 15; losing occurs if the hero’s HP drops to zero at any point. These mechanics underpin all subsequent chapters - the system architecture (Chapter 2) is organized around them, and the AI agents trained in Chapters 3 and 4 learn to navigate them. A concrete description of the game prototype with numerical examples is provided in Introduction Section 2.

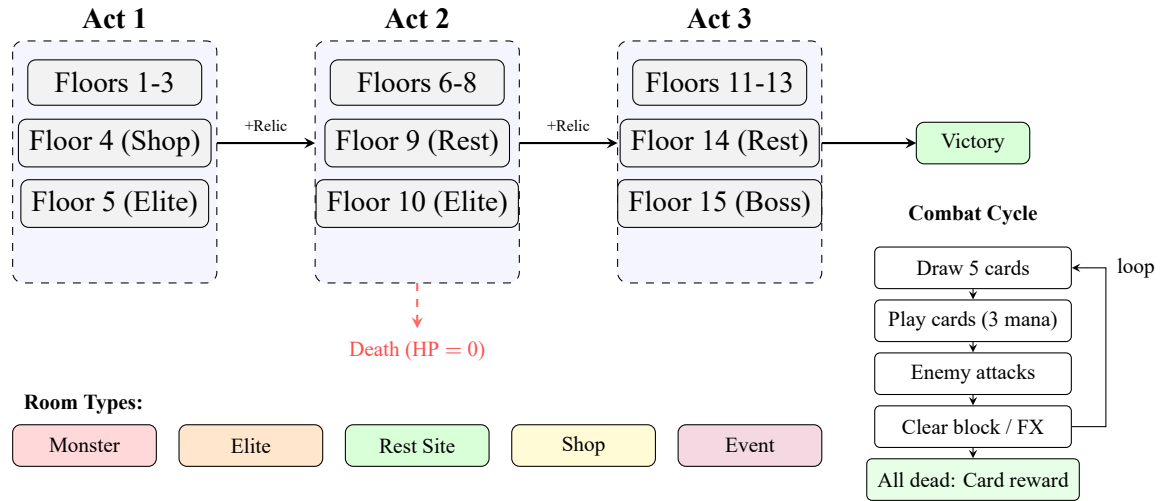


Figure 1.1: Overview of the Roguelike Deckbuilder prototype used in this thesis. A run proceeds through three Acts (15 floors total). Each floor presents one of five room types (legend, bottom left); Monster and Elite rooms trigger the turn-based combat cycle (bottom right), which awards a new card upon victory. The run ends in Victory on defeating the Floor-15 Boss, or Death when HP reaches zero.

1.2 Reinforcement Learning

1.2.1 Why RL for Sub-Problem 2: Intelligent Game Evaluation

Sub-Problem 2 requires agents that can play the game and produce a meaningful quality signal for the optimizer. Several approaches were considered:

Rule-based scripted agents (the Deterministic Heuristic Agent, Section 2.4) are fast and fully deterministic, making them ideal for optimization throughput. However, a fixed rule set cannot adapt to qualitatively different game configurations: if the optimizer creates a configuration where a previously dominant card becomes weak, the DHA’s rules may produce a misleading fitness estimate.

Monte Carlo Tree Search (MCTS) explores future states by sampling random rollout trajectories. While powerful in two-player games such as Go [4], MCTS is computationally prohibitive for our setting: a 15-floor run involves hundreds of sequential decisions, making per-step MCTS impractical within the optimization time budget.

Reinforcement Learning (RL) trains a parameterized policy directly from simulation experience, allowing the agent to learn behavioral strategies from data rather than encoding them manually. RL policies generalize to new configurations if trained with sufficient distribution coverage (addressed via Domain Randomization, Section 1.2.7). By training four RL personas with distinct reward functions, we obtain an evaluation ensemble that collectively approximates a diverse player population, surfacing balance problems that any single evaluator would miss. This ensemble is used as the validation oracle after optimization (Section 4.5).

The following subsections develop the RL theoretical framework that underlies this design: the Markov Decision Process (MDP) formalism, Deep RL, Proximal Policy Optimization (PPO), invalid action masking, and Domain Randomization.

1.2.2 *Markov Decision Processes*

Reinforcement Learning (RL) is the branch of machine learning concerned with training an agent to make sequential decisions in an environment by maximizing a cumulative reward signal [5]. The formal framework is the Markov Decision Process (MDP), defined as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$: state space - the set of all observable configurations of the environment.
- $\mathcal{A}$: action space - the set of possible agent actions.
- $\mathcal{P}(s' \mid s, a)$: transition dynamics - the probability of transitioning to state s' upon taking action a in state s .
- $\mathcal{R}(s, a, s')$: reward function - the immediate scalar signal received after each transition.
- $\gamma \in [0, 1)$: discount factor - controls the relative importance of future versus immediate rewards.

The agent’s behavior is characterized by a policy $\pi(a \mid s)$, which is a probability distribution over actions conditioned on the current state. The objective is to find the optimal policy π^* that maximizes the expected discounted return:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (1.1)$$

1.2.3 Mapping the MDP to Our Game

Before describing specific algorithms, it is important to ground the abstract MDP components in the concrete context of our Roguelike Deckbuilder. This mapping clarifies the design choices made in Chapter 2.

- $\mathcal{S}$ - State space: At each decision point the AI agent observes a 152-dimensional vector encoding the current hero HP, block, and mana; active status effects; the cards in the current hand (up to 10 slots with 7 features each); deck composition; all enemies with their HP, block, and intended next action; equipped relics; and map context (current floor, act, available next rooms). This vector fully captures the information needed for a decision, satisfying the Markov property.
- $\mathcal{A}$ - Action space: The AI agent chooses from 67 discrete actions at each step: play one of up to 10 hand cards (with or without a specific enemy target), end the player turn, navigate to one of the available next-room nodes, use a health potion, or select a card reward. An action mask vector of length 67 marks illegal actions at each step (e.g., playing a card when mana is insufficient).
- $\mathcal{P}(s' \mid s, a)$ - Transition dynamics: The game engine is deterministic given a fixed random seed, but stochastic in practice because card draws, enemy intent patterns, and reward pools are randomized each run. This stochasticity requires the AI agent to plan for uncertainty rather than memorize a fixed sequence of actions.
- $\mathcal{R}(s, a, s')$ - Reward function: Four distinct reward functions are used, one per agent persona. Each shapes behavior toward a specific play style (Aggressive, Defensive, Balanced, Adaptive) through different weightings of events such as enemy kills, HP loss, block gained, and floor progression. The detailed reward functions are defined in Section 2.6.
- γ - Discount factor: We use $\gamma = 0.99$. A 15-floor run involves hundreds of decision steps, so a high discount factor is necessary for the AI agent to reason about late-game

consequences. A low value such as $\gamma = 0.9$ would cause the agent to ignore floor-10 to floor-15 outcomes almost entirely.

1.2.4 Deep Reinforcement Learning

In environments with large or continuous state spaces (such as our game, with a 152-dimensional observation vector), the policy is parameterized by a deep neural network $\pi_\theta(a \mid s)$. The action-value function $Q^\pi(s, a)$ and state-value function $V^\pi(s)$ are defined as:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right] \quad (1.2)$$

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} [Q^\pi(s, a)] \quad (1.3)$$

The advantage function $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ measures how much better action a is compared to the average under policy π .

1.2.5 Proximal Policy Optimization (PPO)

PPO [6] is an on-policy actor-critic algorithm that achieves stable policy updates by constraining the ratio between the updated and old policy:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (1.4)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, $\hat{A}_t$ is the estimated advantage, and ϵ (typically 0.1 - 0.2) is the clipping range. PPO is widely adopted in practice due to its simplicity, sample efficiency relative to trust-region methods, and ease of parallelization [6].

The full PPO objective combines the clipped policy loss with a value function regression loss and an entropy bonus:

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\mathcal{L}^{\text{CLIP}}(\theta) - c_1 \mathcal{L}^{\text{VF}}(\theta) + c_2 H[\pi_\theta](s_t) \right] \quad (1.5)$$

where c_1 and c_2 are coefficients for value loss and entropy bonus, respectively. The entropy term $H[\pi_\theta]$ encourages exploration by penalizing overly deterministic policies.

1.2.6 Invalid Action Masking

In environments with discrete action spaces subject to game-rule constraints, many actions may be illegal at any given step (e.g., playing a card when insufficient mana is available). Training vanilla PPO in such environments wastes sample efficiency as the agent must learn the constraints from scratch.

Invalid Action Masking [7] eliminates illegal actions before policy probability computation by setting the logits of masked actions to $-\infty$ before the softmax:

$$\pi_{\theta}(a \mid s) = \text{softmax}(\mathbf{z} + \mathbf{m}), \quad m_i = \begin{cases} 0 & \text{if action } i \text{ is valid} \\ -\infty & \text{otherwise} \end{cases} \quad (1.6)$$

This approach is theoretically equivalent to valid policy gradient updates [7] and dramatically accelerates learning in constraint-heavy environments such as deckbuilding games. The *sb3-contrib* library provides *MaskablePPO*, a drop-in extension of Stable-Baselines3 [8] PPO that natively supports action masking.

1.2.7 Domain Randomization

Domain Randomization (DR) was originally proposed to transfer robot policies trained in simulation to the physical world [9]. The key insight is that if a policy is trained across a distribution of simulation environments (parameterized by varying physics, lighting, or object properties), it may generalize to the real world as merely another sample from that distribution.

In our context, DR serves a different but analogous purpose: because the NSGA-II optimization modifies game parameters across a bounded range, the RL agents must remain competent across all configurations in that range, not just the nominal one. At each environment *reset()*, we add uniform noise $\epsilon \sim \mathcal{U}(-\delta, \delta)$ to each game parameter:

$$\tilde{\theta}_i = \theta_i \cdot (1 + \epsilon_i), \quad \epsilon_i \sim \mathcal{U}(-\delta, \delta) \quad (1.7)$$

This forces agents to learn behavioral policies based on relative relationships (e.g., “play the highest-damage card relative to enemy HP”) rather than memorizing absolute parameter values, making them robust evaluators for any point in the optimization search space.

1.3 Optimization Algorithms

This section addresses Sub-Problem 3: automated search over the game parameter space for configurations that simultaneously satisfy the Balance, Engagement, and Coherence objectives. We first survey the landscape of automated game balancing approaches to motivate the choice of evolutionary methods, then present the single-objective Genetic Algorithm (Section 1.3.2) and the multi-objective NSGA-II (Section 1.3.3).

1.3.1 Survey of Automated Game Balancing Approaches

Before introducing the specific algorithms used in this thesis, it is useful to survey the landscape of automated game balancing approaches. This positions the choice of evolutionary algorithms and makes explicit why simpler alternatives are insufficient for Sub-Problem 3.

Manual playtesting is the baseline industry approach: designers adjust parameters by hand based on player feedback. While it captures nuanced human perception, it does not scale - a 500-parameter space requires exponentially many sessions to explore systematically, and each iteration takes days or weeks of design time.

Hill-climbing and local search start from an initial configuration and iteratively perturb one parameter at a time, accepting changes that improve a scalar fitness score. These methods are easy to implement but are highly susceptible to local optima in non-convex, high-dimensional landscapes, and they cannot handle competing objectives without collapsing them into a single weighted sum.

Simulated annealing extends hill-climbing with a temperature schedule that occasionally accepts worse solutions, enabling escape from local optima. However, it still requires a single scalar fitness function, losing the trade-off information between Balance, Engagement, and Coherence that a Pareto front preserves.

Coevolutionary methods evolve both game parameters and agent strategies simultaneously, using inter-agent competition dynamics as a balance signal [10]. While theoretically elegant, coevolution is notoriously unstable: the fitness landscape shifts as agents co-adapt, making convergence unreliable without carefully tuned population sizes and update schedules.

Evolutionary Algorithms (EA) address the key shortcomings above. By maintaining a *population* of candidate solutions, EAs explore the search space globally rather than locally; crossover recombines partial solutions from fit individuals to discover configurations that neither parent achieves alone; elitism preserves the best solutions found. For our 500-dimensional continuous parameter space with three competing objectives, EAs are the most suitable class

of methods. Section 1.3.2 presents the single-objective GA; Section 1.3.3 extends this to the multi-objective NSGA-II, which returns a Pareto front rather than a single solution.

1.3.2 Genetic Algorithms

Genetic Algorithms (GA) are a family of population-based metaheuristics that abstract the mechanisms of biological evolution [11]. A GA maintains a population of candidate solutions (genomes), evaluates their fitness with respect to an objective function, and iteratively applies selection, crossover, and mutation operators to produce progressively better populations. The key insight is that combining partial solutions from fit individuals (crossover) can discover configurations that neither parent achieves alone, while random perturbations (mutation) prevent premature convergence to local optima.

Core Components

Genome representation encodes a candidate solution as a chromosome. For continuous optimization problems, genomes are vectors of real numbers $\mathbf{g} = (g_1, g_2, \dots, g_n) \in \mathbb{R}^n$. In our game-balancing application, each gene g_i is a multiplier applied to a specific game parameter (e.g., the base damage of a particular card).

Fitness function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ assigns a scalar quality score to each genome. For single-objective GA:

$$F(\mathbf{g}) = w_1 \cdot f_{\text{balance}}(\mathbf{g}) + w_2 \cdot f_{\text{engagement}}(\mathbf{g}) + w_3 \cdot f_{\text{coherence}}(\mathbf{g}) \quad (1.8)$$

where w_1, w_2, w_3 are pre-determined weights. In our game-balancing application (Sub-Problem 3), each term maps directly to a simulation-derived quantity:

- $f_{\text{balance}}(\mathbf{g})$: a score in $[0, 1]$ measuring how closely the AI agent win rate, average victory HP, and average floor-on-death align with designer-specified targets (45%, 30%, and Floor 10, respectively). It is computed by the *FitnessEvaluator* from batch simulation statistics.
- $f_{\text{engagement}}(\mathbf{g})$: a score in $[0, 1]$ measuring the fraction of cards that are genuinely viable - i.e., picking them correlates positively with winning. A configuration where only 3 out of 30 cards are worth picking scores near 0; one where all 30 are competitive scores near 1.
- $f_{\text{coherence}}(\mathbf{g})$: a score in $[0, 1]$ penalizing trap cards (high pick rate but below-average win contribution) and parameter values outside an interpretable range. A configuration with no trap cards and all scaling multipliers in $[0.5, 2.0]$ scores near 1.

- $w_1 = 0.5$, $w_2 = 0.3$, $w_3 = 0.2$: designer-chosen weights reflecting that balance is the primary concern, followed by engagement and coherence. These weights are fixed before optimization, which is a key limitation of the single-objective GA addressed by NSGA-II in Section 1.3.3.

Selection determines which genomes reproduce. Tournament selection of size k draws k individuals uniformly at random and returns the fittest, providing tunable selection pressure without requiring global fitness ranking.

Crossover combines two parent genomes to produce offspring. Uniform crossover assigns each gene to offspring 1 or offspring 2 with probability 0.5, effectively sampling a random hypercube between the two parents:

$$g_i^{\text{child}} = \begin{cases} g_i^{P_1} & \text{with probability 0.5} \\ g_i^{P_2} & \text{with probability 0.5} \end{cases} \quad (1.9)$$

Mutation perturbs genes stochastically to maintain diversity. Gaussian mutation adds zero-mean noise:

$$g'_i = g_i + \mathcal{N}(0, \sigma^2), \quad \text{with probability } p_m \quad (1.10)$$

Typical values are $p_m \in [0.01, 0.05]$ and $\sigma \in [0.05, 0.2]$ for normalized parameter ranges.

Elitism preserves a fraction of the top individuals unchanged across generations, ensuring that the best discovered solution never degrades.

1.3.3 NSGA-II: Multi-Objective Evolutionary Optimization

While the GA (Section 1.3.2) uses a fixed weighted sum of objectives, this collapses the multi-dimensional trade-off space into a single number determined by the designer's prior weighting. NSGA-II addresses this limitation by returning a full Pareto front of non-dominated solutions in a single run, giving the designer a post-hoc menu of configurations to choose from.

Why Multi-Objective Optimization for Game Balancing?

Several paradigms exist for optimizing problems with multiple competing objectives. **Weighted-sum scalarization** (used by the single-objective GA, Section 1.3.2) converts all objectives into a single scalar; it is computationally cheap but requires committing to fixed weights w_i before optimization, which pre-determines the trade-off without seeing the actual design space. **ε -constraint methods** optimize one objective while bounding the others as inequality

constraints; each run produces a single Pareto-optimal point, requiring k runs to trace k points on the front. **Decomposition-based methods** (e.g., MOEA/D) decompose the problem into scalar subproblems along a grid of weight vectors; they are efficient but still require the designer to pre-specify the weight grid. **Pareto-based evolutionary methods** - of which NSGA-II is the canonical example - maintain a population of non-dominated solutions and return the full Pareto front in a single run; they are the natural choice when the designer’s preference ordering is unknown a priori.

For our game-balancing problem, a Pareto-based approach is preferred for two reasons. First, no weight vector can fully capture designer intent across all possible game audiences: a configuration targeting casual players prioritizes engagement over strict balance, while a competitive-mode configuration inverts these priorities. Second, revealing the actual trade-off curve allows the designer to make an informed post-hoc selection - a qualitatively richer outcome than a single number.

A straightforward alternative to NSGA-II is a weighted-sum fitness function: $F = w_1 \cdot \text{Balance} + w_2 \cdot \text{Engagement} + w_3 \cdot \text{Coherence}$. This is exactly what the single-objective GA in this thesis implements with weights $(0.5, 0.3, 0.2)$. The limitation is that the weights must be chosen before optimization runs, but no single weight vector is universally correct: a game designer targeting tournament play might prioritize balance above all, while one targeting casual players might prefer high engagement even at a small cost to balance.

NSGA-II avoids this limitation by treating all three objectives as separate criteria and returning a *Pareto front* - a set of solutions where improving one objective necessarily degrades at least one other. The designer inspects this front *after* seeing the actual trade-off space and selects the solution matching their priorities. This post-hoc selection is far more informative than blindly pre-specifying weights.

A second advantage is search diversity: NSGA-II’s crowding distance mechanism actively maintains a spread of solutions across the Pareto front, preventing the population from collapsing onto a single region of the objective space. In game balancing, where the interaction between Balance, Engagement, and Coherence is non-linear and hard to predict, this diversity is essential for discovering unexpected high-quality configurations.

Multi-Objective Problem Formulation

Many real-world optimization problems require simultaneously maximizing several objectives that are in mutual tension [12, 13]. The general multi-objective optimization problem

is:

$$\text{maximize } \mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x})) \quad (1.11)$$

Because improving one objective typically harms another, there is no single “optimal” solution. Instead, the goal is to characterize the Pareto front - the set of solutions that are non-dominated.

Definition 1.1 (Pareto Dominance). *Solution $\mathbf{x}_1$ dominates $\mathbf{x}_2$ (written $\mathbf{x}_1 \prec \mathbf{x}_2$) if and only if:*

$$\forall i : f_i(\mathbf{x}_1) \leq f_i(\mathbf{x}_2) \quad \text{and} \quad \exists j : f_j(\mathbf{x}_1) < f_j(\mathbf{x}_2) \quad (1.12)$$

Definition 1.2 (Pareto Optimal Set). *A solution $\mathbf{x}^*$ is Pareto optimal if no solution in the feasible space dominates it. The set of all Pareto optimal solutions forms the Pareto front.*

The NSGA-II Algorithm

NSGA-II (Non-dominated Sorting Genetic Algorithm II), proposed by Deb et al. [12], is the canonical multi-objective evolutionary algorithm. Its three key innovations over the original NSGA are: (1) efficient $\mathcal{O}(MN^2)$ fast non-dominated sorting, where M is the number of objectives and N is the population size; (2) a crowding distance metric for diversity preservation; and (3) an elitist selection strategy.

Fast non-dominated sorting partitions the population P into fronts $\mathcal{F}_1, \mathcal{F}_2, \dots$ where $\mathcal{F}_1$ contains all non-dominated individuals, $\mathcal{F}_2$ those dominated only by members of $\mathcal{F}_1$, and so on. The rank of individual i equals its front index.

Crowding distance d_i measures the density of solutions surrounding individual i in objective space:

$$d_i = \sum_{k=1}^m \frac{f_k^{(i+1)} - f_k^{(i-1)}}{f_k^{\max} - f_k^{\min}} \quad (1.13)$$

where $i \pm 1$ denotes the adjacent individuals when sorted by objective k . Boundary solutions receive $d_i = \infty$.

NSGA-II selection operator $\prec_n$ prefers lower rank, breaking ties by larger crowding distance:

$$\mathbf{x}_1 \prec_n \mathbf{x}_2 \iff \text{rank}(\mathbf{x}_1) < \text{rank}(\mathbf{x}_2) \quad \text{or} \quad (\text{rank}(\mathbf{x}_1) = \text{rank}(\mathbf{x}_2) \text{ and } d_1 > d_2) \quad (1.14)$$

This guarantees convergence toward the Pareto front while maintaining diversity along it.

Simulated Binary Crossover (SBX) and Polynomial Mutation are the preferred genetic operators for real-valued chromosomes in NSGA-II. SBX mimics single-point crossover for

binary representations but operates directly on floating-point variables [12].

1.4 Related Work

1.4.1 Automated Game Balancing

Nelson and Mateas [10] proposed a broader framework for automated game design, though primarily for simple rule-based games.

Our work distinguishes itself from this body of literature by (1) addressing the more complex Deckbuilder sub-genre, (2) employing four learned RL personas instead of scripted evaluation bots, and (3) providing a systematic empirical comparison between a flat single-objective GA and a structured multi-objective NSGA-II under identical simulation budgets.

1.4.2 RL for Card Games

Reinforcement learning has been successfully applied to complex card games. AlphaStar demonstrated superhuman performance in StarCraft II using a combination of imitation learning and self-play [14]. The foundational DQN algorithm [15] has since been applied to discrete card-game environments such as Dominion, while PPO-family algorithms have been explored in simplified Slay the Spire variants. Both lines of work confirm that deep RL is well suited to the discrete, constraint-heavy action spaces of card games, but neither connected agent training to systematic game balancing - a gap this thesis directly addresses.

1.4.3 Procedural Content Generation

Togelius et al. [16] provide a comprehensive survey of search-based PCG, demonstrating that evolutionary approaches are effective at generating game content that satisfies multiple quality criteria. Preuss et al. [17] apply multi-objective optimization specifically to PCG, a setting directly analogous to our balancing problem. Our contribution complements this line of work by focusing on parameter tuning (rather than content generation) and by using RL agents as the evaluation oracle.

CHAPTER 2

SYSTEM ANALYSIS AND DESIGN

This chapter describes how the system is designed to meet the research objectives stated in the Introduction. It proceeds top-down: Section 2.1 defines the functional and non-functional requirements derived from the game mechanics introduced in the Introduction; Section 2.2 presents the four-layer architecture that organizes the system and connects each layer to those requirements; Sections 2.3 to 2.7 then detail the design of each major component - the game engine, the heuristic AI agent, the RL environment, the four AI agent personas, and the optimization layer.

2.1 Requirements Analysis

The three sub-problems identified in the Introduction translate directly into system requirements. Sub-Problem 1 (Core Game Engine) drives requirements for the simulation, data representation, and heuristic baseline (FR1-FR2). Sub-Problem 2 (AI Agent for Evaluation) drives requirements for the RL environment and agent training pipeline (FR3-FR4). Sub-Problem 3 (Multi-Objective Optimization) drives requirements for the evolutionary optimizer and output visualization (FR5-FR6). Each requirement below is grounded in the theory established in Chapter 1 and cross-referenced to the research objective it fulfills.

2.1.1 Functional Requirements

The functional requirements are derived directly from the game mechanics described in the Introduction and the research objectives stated there. Each requirement corresponds to a concrete capability the system must provide.

1. FR1 - Game Engine: Implement a complete, self-contained Roguelike Deckbuilder game engine in Python supporting: turn-based combat with a card-based attack system (as described in Introduction Section 2.1), a procedurally generated 15-floor room map, deck acquisition and management, relic collection, the five room types (Monster, Elite, Event, Shop, Rest), and a data-driven parameter system stored in JSON. This directly supports Objective 1.

2. FR2 - Deterministic Baseline AI Agent: Provide a rule-based AI agent that can play the game to completion, serving as a playability validator and initial balancing oracle. This directly supports Objective 2.
3. FR3 - RL Environment: Expose the game engine as an OpenAI Gymnasium-compatible environment with a well-defined observation space, action space, and action masking mechanism, enabling AI agent training. This supports Objective 3.
4. FR4 - RL AI Agent Training: Train four MaskablePPO AI agents with distinct reward functions and produce checkpoints that can be loaded for evaluation. This directly supports Objective 3.
5. FR5 - Game Parameter Optimization: Implement both a single-objective GA and a multi-objective NSGA-II that use AI agent evaluation to search for balanced parameter configurations. This directly supports Objective 4.
6. FR6 - Results Visualization: Generate evolution charts, Pareto front visualizations, and card viability analyses. This supports Objective 5.

2.1.2 *Non-Functional Requirements*

1. NFR1 - Cross-platform Compatibility: The game engine must run on Linux to be usable inside Kaggle kernels for GPU training. A pure Python implementation satisfies this requirement natively without any platform-specific build steps or language bridges.
2. NFR2 - Simulation Speed: The game engine must support sufficient throughput to evaluate each optimizer generation (population size 50, 20 runs per genome = 1,000 simulations) within a practical wall-clock budget on Kaggle hardware.
3. NFR3 - Reproducibility: Experiments must be reproducible via fixed random seeds.
4. NFR4 - Extensibility: New cards, relics, and enemy types must be addable by modifying only the JSON data files without changing source code.

2.2 System Architecture

The architecture is designed so that each layer directly addresses one or more sub-problems from the Introduction and is grounded in the theory developed in Chapter 1. The Core Game Layer satisfies Sub-Problem 1 using the game design principles from Section 1.1. The Simulation/Training Layer satisfies Sub-Problem 2, implementing the MaskablePPO environment

and training pipeline whose theoretical basis is Section 1.4. The Optimization Layer satisfies Sub-Problem 3, deploying the GA (Section 1.2) and NSGA-II (Section 1.3) over the hierarchical genome. The Data Layer ties all layers together through the shared *GameData.json* parameter interface that makes the optimization target fully separable from game logic.

The system is organized into four principal layers, as illustrated in Figure 2.1. Each layer maps directly to one or more functional requirements.

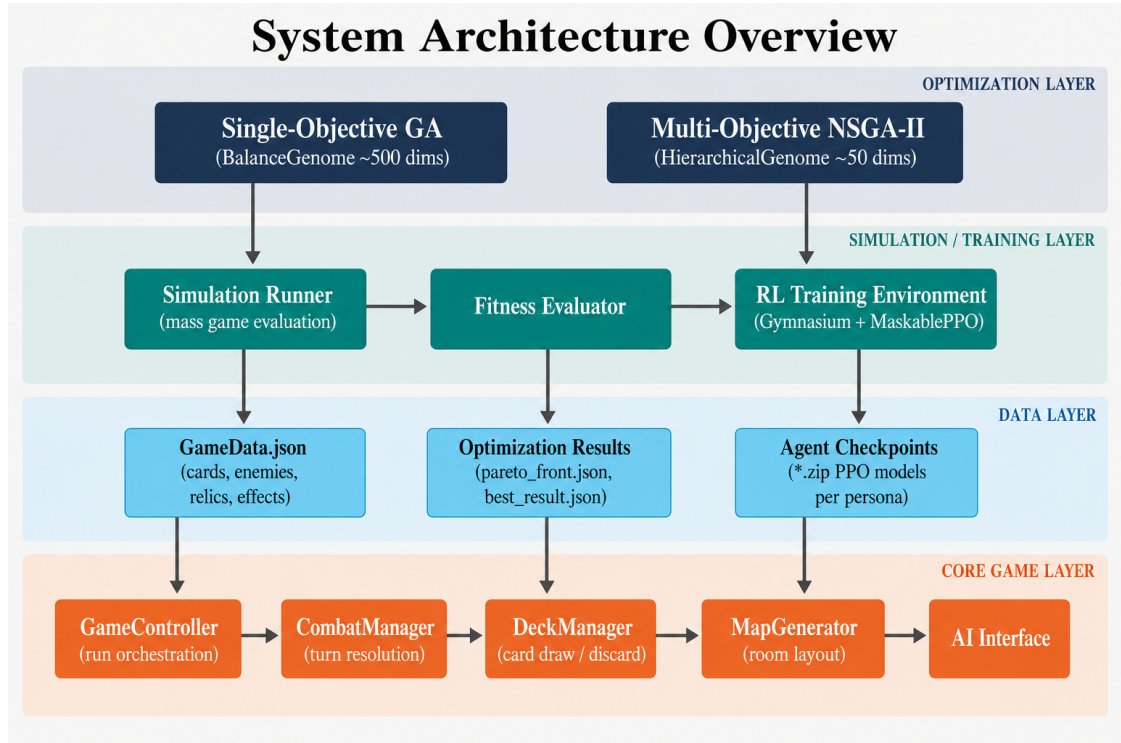


Figure 2.1: High-level system architecture showing data flow between the Optimization Layer, Simulation/Training Layer, Data Layer, and Core Game Layer.

- **Core Game Layer** (fulfills FR1, FR2): Contains the five components shown in the diagram - *GameController* (run orchestration), *CombatManager* (turn resolution), *DeckManager* (card draw and discard), *MapGenerator* (room layout), and *AI Interface* (the common interface all AI agents implement). This layer is deterministic given a fixed random seed and a fixed parameter set θ , which is essential for reproducible fitness evaluation.
- **Data Layer** (fulfills FR1, FR5, FR6): Three data stores sit between the game and the upper layers. *GameData.json* defines all game entities - cards, enemies, relics, and status effects. *Optimization Results* stores the output of each optimizer run (*pareto_front.json*, *best_result.json*). *Agent Checkpoints* stores the trained PPO model files (one .zip per persona). Separating data from logic means the optimization layer can modify parameters without recompiling any code.

- **Simulation/Training Layer** (fulfills FR3, FR4): Three components operate here. The *Simulation Runner* executes mass game runs to compute fitness metrics for the optimization layer. The *Fitness Evaluator* aggregates raw simulation statistics into the three objective scores (Balance, Engagement, Coherence). The *RL Training Environment* wraps the game engine as a Gymnasium environment with MaskablePPO support for AI agent training.
- **Optimization Layer** (fulfills FR5): Two optimizers operate at the top level. A lightweight inline NSGA-II (used in Kaggle notebooks) evaluates populations of 30 genomes over 15 generations. The primary *Multi-Objective NSGA-II* via pymoo uses the *HierarchicalGenome* representation (38 continuous dimensions) with a population of 50 over 30 generations. Both send candidate genomes down to the Simulation Runner and receive fitness scores back.

2.3 Core Game Engine Design

2.3.1 Game Structure Overview

A single game run consists of 15 floors organized into three Acts of five floors each. The player navigates a branching map of rooms, engaging in combat, collecting cards and relics, and managing resources (HP, gold, mana). The game ends in victory if the player defeats the final boss on Floor 15, or in defeat if HP drops to zero.

Map generation uses a constrained random algorithm that guarantees at least one viable path from the entrance to the exit floor while distributing room types according to configurable weights. A typical 15-floor map contains: 8 - 10 Monster rooms, 2 - 3 Elite rooms, 1 - 2 Shop rooms, 2 - 3 Rest Sites, and 2 - 3 Event rooms.

Card system: The game contains over 30 distinct cards organized by type (Attack, Skill, Power) and star rating (1 - 5). Cards have a mana cost, and each round the player draws five cards, spending mana to play them. Cards are acquired as rewards after combat and can be removed or upgraded at rest sites or via events.

Relic system: Relics are passive items collected from Elite rooms, events, and shops. Over 20 relics are defined, each with a trigger condition (e.g., *OnEnemyAttack*, *OnTurnStart*) and an effect (e.g., add thorns, reduce mana cost, draw extra cards).

2.3.2 Combat System Design

The combat system follows a turn-based structure with alternating player and enemy phases:

1. Player turn: The player draws 5 cards, then plays any combination of affordable cards using available mana (default: 3 mana per turn). Actions include dealing damage to a target enemy, gaining block (temporary defense), applying status effects, and drawing additional cards.
2. Enemy turn: Each enemy executes its scheduled action (attack, buff, or debuff) according to a pre-defined pattern. Status effects (Strength, Weakness, Poison, Burn, Vulnerability) are resolved.
3. End-of-turn: Block is cleared, effects are ticked down, and the player draws a new hand.

Combat resolves when all enemies are defeated (player wins the room) or the player's HP reaches zero (run ends). The class diagram in Figure 2.2 shows the relationship between the key combat classes.

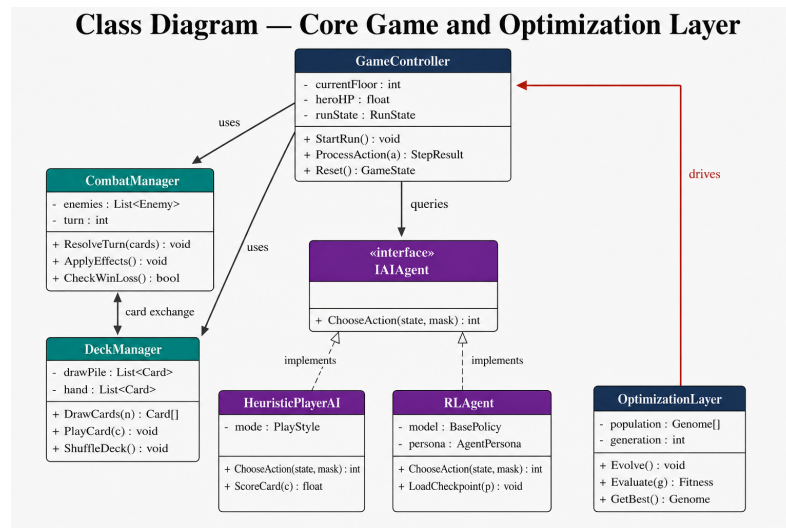


Figure 2.2: Class diagram of the core game module showing the relationships between *GameController*, *CombatManager*, *DeckManager*, the AI interface, and the optimization layer.

Figure 2.3 illustrates the sequence of interactions during a single combat turn.

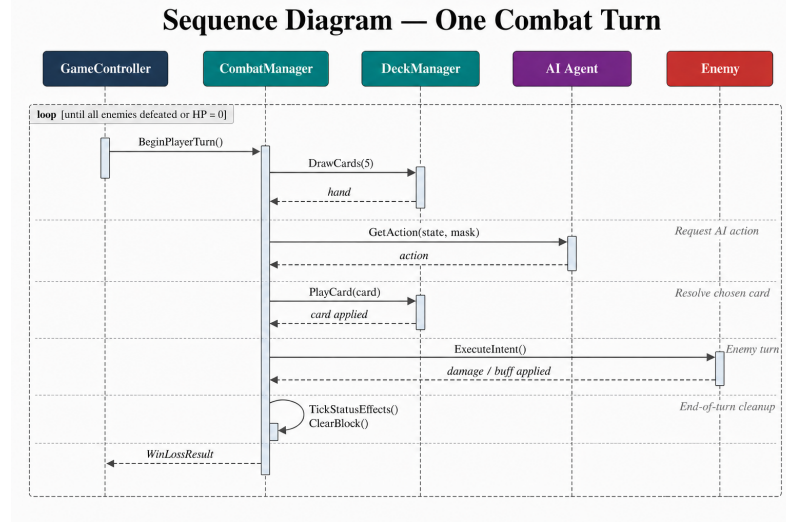


Figure 2.3: Sequence diagram of one combat turn: the Combat Manager draws cards from the Deck Manager, queries the AI Agent for a decision, resolves the card action, executes the enemy turn, and checks win/loss conditions.

2.4 Deterministic Heuristic Agent Design

2.4.1 Purpose and Role

The Deterministic Heuristic Agent (DHA), implemented as *HeuristicPlayerAI*, serves two specific roles in the project:

1. Playability validation: Before any optimization or RL training, the DHA confirms that the game is winnable and that no trivially broken configurations exist. A game that a reasonable agent cannot win at all (or can always win trivially) is not ready for balancing.
2. Optimization benchmark: In the initial GA/NSGA-II experiments documented in Section 4.4, the DHA provides a consistent, fast, and reproducible evaluation signal. Because the DHA executes in microseconds per decision (no neural network inference), it enables the optimization algorithm to run orders of magnitude more simulations than would be possible with RL agents.

2.4.2 Decision Logic

The DHA implements a three-level priority hierarchy for in-combat decisions:

1. Survival first: Calculate total incoming damage from all enemies in the current turn. If incoming damage exceeds current HP, preferentially play defensive cards (Block, Shielding

skills) to survive.

2. Lethal second: If any single target can be killed this turn with available cards and mana, prioritize those cards to reduce enemy damage output.
3. Efficient offense third: Otherwise, play the card with the highest damage-per-mana ratio, preferring multi-target attacks when multiple enemies are present.

For map navigation, the DHA scores each available next-room type based on current HP percentage: it seeks Elite rooms when $HP \geq 60\%$ (rewarded with rare cards and relics), Rest sites when $HP \leq 40\%$, and Monster rooms otherwise. Card rewards are chosen by star rating, preferring higher rarities and maintaining a balanced ratio of attack to skill cards.

The DHA’s deterministic nature makes it an ideal baseline: given the same game seed and parameters, it always makes the same choices, ensuring that measured fitness differences between parameter sets are attributable to the parameters rather than agent stochasticity.

2.5 Reinforcement Learning Environment Design

2.5.1 Observation Space

The observation vector $\mathbf{s} \in \mathbb{R}^{152}$ encodes the full game state at the moment the agent must make a decision. Table 2.1 describes its components.

All values are normalized to $[0, 1]$ to stabilize neural network training. Unknown or empty slots are filled with zeros (e.g., empty hand slots, absent enemies).

2.5.2 Action Space

The action space is discrete with 67 actions, organized by game phase:

- Action 0: End player turn.
- Actions 1 - 40: Play hand-card slot h targeting enemy slot e , encoded as $h \times 4 + e + 1$ for $h \in [0, 9], e \in [0, 3]$.
- Actions 41 - 47: Navigate to map node index 0 - 6.
- Actions 48 - 50: Choose event option 0 - 2.
- Actions 51 - 55: Pick card reward slot 0 - 3, or skip (slot 4).

Table 2.1: Observation space components (total: 152 dimensions).

Component	Dim.	Description
Hero state	4	HP ratio, block ratio, mana ratio, gold ratio
Active effects	7	Strength, Weakness, Poison, Burn, Vulnerability, Thorns, Metallicize
Hand cards	70	10 slots $\times$ 7 features (cost, star, type, #actions, affordable, dmg, block)
Deck star counts	5	Count of 1-star through 5-star cards in master deck
Enemies	24	4 slots $\times$ 6 features (HP, block, star, #effects, intent type, intent value)
Relics	10	10 slots: equipped relic star ratings (normalized)
Game state (one-hot)	7	<i>PreRun, OnMap, InCombat, InEvent, InShop, AwaitingReward, GameOver</i>
Next map nodes	14	7 slots $\times$ 2 features (room type, star rating)
Contextual features		
Current floor	1	floor / 16
Combat turn	1	turn_number / 20
Draw pile size	1	#cards in draw pile / 30
Total incoming dmg	1	Sum of enemy intent damage / 60
Affordable hand dmg	1	Max damage playable with current mana / 50
Turns counter	1	turn_number / 10
Deck attack ratio	1	Fraction of Attack cards in master deck
Cards played (turn)	1	Cards played this turn / 5
Affordable block	1	Max block playable with current mana / 20
Block vs intent	1	Current block / incoming damage (capped at 1)
Floor (repeated)	1	floor / 16 (for convenience indexing)
Total	152	

- Actions 56 - 60: Buy shop card slot 0 - 4.
- Actions 61 - 65: Buy shop relic slot 0 - 4.
- Action 66: Leave shop.

At each step, the environment returns an action mask - a binary vector $\mathbf{m} \in \{0, 1\}^{67}$ - indicating which actions are currently legal. Illegal actions (e.g., playing a card with insufficient mana, navigating to an unavailable node) always have mask value 0.

2.5.3 Episode Structure

One episode corresponds to one complete game run from Floor 1 to either Floor 15 victory or death. The environment *reset()* starts a new run (optionally applying Domain Randomization), and *step(action)* advances the game state by one decision, returning $(s', r, \text{done}, \text{info})$.

2.6 RL Agent Persona Design

2.6.1 Design Rationale

Using a single agent persona for game evaluation has a significant limitation: it may only reveal balance problems that matter to one specific play style. A defensive specialist might expose different vulnerabilities than an aggressive burn-damage specialist. By training four distinct personas, we obtain a richer evaluation signal that better approximates the diversity of real players.

Each persona shares the same observation space (152-dim) and action space (67 actions), but is shaped by a distinct reward function that incentivizes a specific behavioral style. The neural network architecture for all agents is a two-hidden-layer MLP with layers of size [512, 512] for both the actor and critic.

2.6.2 Aggressive Agent

Behavioral goal: Minimize the number of turns to defeat enemies. Accept HP loss in exchange for faster kills. Prefer high-damage cards over defensive ones.

Reward function:

$$\begin{aligned}
r_t^{\text{agg}} = & r_{\text{base}} - 0.10 \cdot \mathbb{I}[\text{step taken}] - 0.10 \cdot |\Delta\text{HP}_{\text{lost}}| \cdot m(\text{HP}\%) \\
& + 5.0 \cdot \mathbb{I}[\text{enemy killed}] + 500.0 \cdot \mathbb{I}[\text{victory}] \\
& + 15.0 \cdot \left(1 - \frac{t}{t_{\text{max}}}\right) \cdot \mathbb{I}[\text{victory}] - 200.0 \cdot \mathbb{I}[\text{death}]
\end{aligned} \tag{2.1}$$

where $m(\text{HP}\%) = 3$ if $\text{HP}\% < 0.25$, 2 if $\text{HP}\% < 0.40$, 1 otherwise. The step penalty -0.10 drives the agent to end combats quickly. The HP loss penalty scales with danger: $3\times$ below 25% HP and $2\times$ below 40% HP (a “non-linear brake”), to prevent suicidal behavior while still permitting calculated risk-taking at full HP. The speed bonus on victory rewards efficient play.

2.6.3 Defensive Agent

Behavioral goal: Survive as long as possible, prioritizing healing and block generation. Sacrifice offensive tempo to preserve HP.

Reward function:

$$\begin{aligned}
r_t^{\text{def}} = & r_{\text{base}} - 0.005 \cdot \mathbb{I}[\text{step taken}] + 0.30 \cdot \Delta\text{HP}_{\text{healed}} - 0.15 \cdot \min(\Delta\text{HP}_{\text{lost}}, 20) \\
& + 0.30 \cdot \Delta\text{Block} + 2.0 \cdot \mathbb{I}[\text{combat survived}] \\
& + 500.0 \cdot \mathbb{I}[\text{victory}] + 80.0 \cdot \frac{\text{HP}_{\text{final}}}{\text{HP}_{\text{max}}} \cdot \mathbb{I}[\text{victory}] - 200.0 \cdot \mathbb{I}[\text{death}]
\end{aligned} \tag{2.2}$$

The asymmetric HP weights ($+0.30$ for healing vs. -0.15 for damage) create a strong preference for rest sites and block cards. The HP-loss cap at 20 per step prevents gradient instability when the agent receives a large boss hit. The very small step penalty (-0.005) means the agent is not penalized for taking many turns to kill enemies. The terminal win bonus is proportional to remaining HP, further incentivizing conservative play.

2.6.4 Balanced Agent

Behavioral goal: Represent a standard player who weighs offense and defense equally, manages gold efficiently, and aims for a moderate win rate.

Reward function:

$$\begin{aligned}
r_t^{\text{bal}} = & r_{\text{base}} + 0.20 \cdot \Delta\text{HP}_{\text{healed}} - 0.20 \cdot \Delta\text{HP}_{\text{lost}} \\
& + 0.02 \cdot \Delta\text{gold} + 500.0 \cdot \mathbb{I}[\text{victory}] + 30 \cdot \left(\frac{\text{HP}_{\text{final}}}{\text{HP}_{\text{max}}} \right)^2 \cdot \mathbb{I}[\text{victory}] - 200.0 \cdot \mathbb{I}[\text{death}]
\end{aligned} \tag{2.3}$$

The symmetric HP weights model a player who is neither reckless nor overly cautious. The flat win bonus of +500 provides a strong terminal signal, while the quadratic HP bonus further incentivizes the agent to win with as much HP remaining as possible, serving the balancing objective of “victory HP.”

2.6.5 Adaptive Agent

Behavioral goal: Dynamically switch between aggressive and defensive behaviors based on current HP, without being explicitly programmed to do so.

Design challenge: An earlier version of this agent used a reward function that explicitly changed its formula based on HP ratio at runtime (high HP $\rightarrow$ aggressive formula; low HP $\rightarrow$ defensive formula), which is non-stationary and caused the value function to diverge. The final design (v7) resolves this by ensuring that every reward signal is a function of observable state features that are also present in the 152-dimensional observation vector, preserving the Markov property.

Final implementation (v7): The Adaptive Agent uses an HP-state-aware reward function:

$$\begin{aligned}
r_t^{\text{ada}} = & r_{\text{base}} - 0.01 \cdot \mathbb{I}[\text{step taken}] - (0.1 + 0.3 \cdot (1 - \text{HP}\%)) \cdot |\Delta\text{HP}_{\text{lost}}| \\
& + 2.0 \cdot \mathbb{I}[\text{enemy killed}] + 25.0 \cdot \mathbb{I}[\text{floor advanced}] \\
& + 500.0 \cdot \mathbb{I}[\text{victory}] - 200.0 \cdot \mathbb{I}[\text{death}]
\end{aligned} \tag{2.4}$$

The HP loss penalty scale $(0.1 + 0.3 \cdot (1 - \text{HP}\%))$ increases continuously as HP decreases, reaching 0.4 at 0% HP. This creates a smooth, gradient-friendly incentive for the agent to treat HP differently depending on its current value - a behavior that is fully MDP-consistent since $\text{HP}\% = s[0]$ is the first element of the observation vector. The floor advance bonus (+25.0 per floor cleared) drives progression even under non-combat HP pressure, while the idle stagnation penalty (-0.05 per step after 400 steps on the same floor) prevents the agent from grinding indefinitely.

2.7 Optimization Layer Design

2.7.1 Data Flow

Figure 2.4 shows how data flows from the optimization layer through simulation to fitness evaluation.

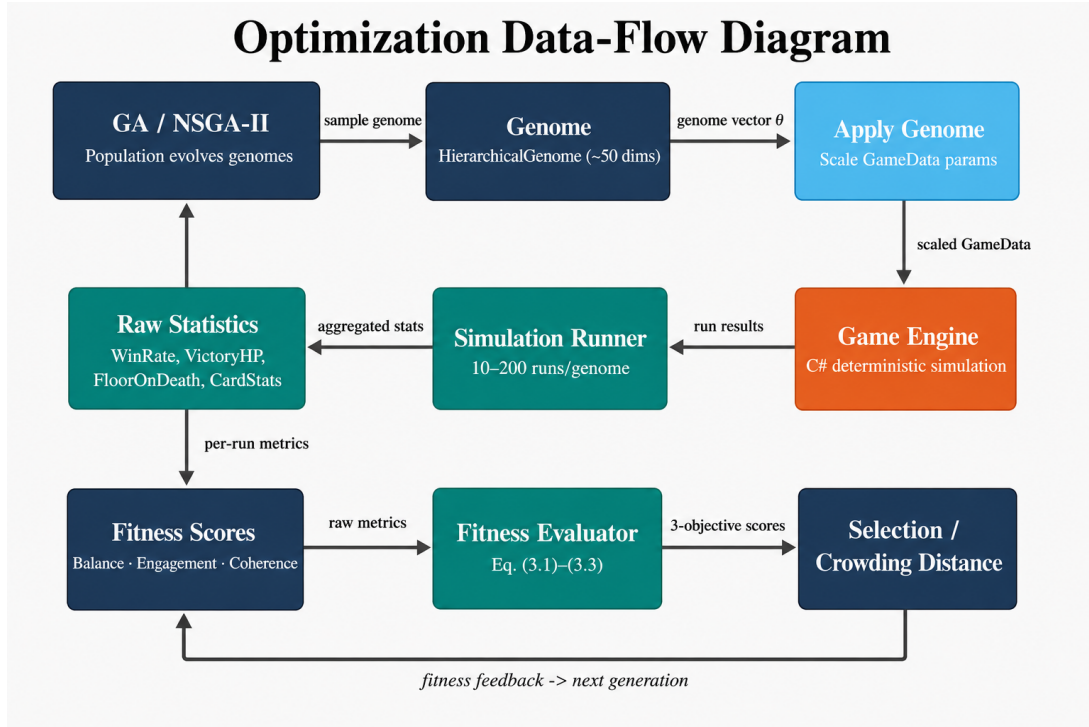


Figure 2.4: Data flow in the optimization loop. A genome is created by the optimization layer, applied to a scaled *GameData* copy, passed to the simulation runner which uses the AI agent to play games, and resulting statistics are evaluated into fitness scores that feed back to the optimizer.

2.7.2 Genome Representations

BalanceGenome (Single-Objective GA)

The *BalanceGenome* directly encodes one scalar multiplier per game parameter, resulting in a chromosome of approximately 500 dimensions. The genome consists of four dictionaries:

- *CardActionScalars*: One float per card action value (damage or block amount per card).
- *CardCostModifiers*: Integer adjustments to each card’s mana cost.
- *EnemyHealthScalars*: One float per enemy type scaling base HP.
- *EnemyActionValueScalars*: One float per enemy action scaling attack damage.

This flat representation allows fine-grained per-entity adjustment but suffers from the curse of dimensionality and tends to produce incoherent solutions where logically related parameters (e.g., all 1-star cards) receive unrelated scalars.

HierarchicalGenome (NSGA-II)

The *HierarchicalGenome* reduces the search space to 38 continuous dimensions by grouping parameters into semantically meaningful categories:

- Global multipliers (5 params): Overall scaling for damage, health, block, mana cost, gold rewards.
- Progression scaling (9 params): Separate damage, health, and block multipliers for Early (floors 1 - 5), Mid (floors 6 - 10), and Late (floors 11 - 15) game phases.
- Card type and star scalars (8 params): Per-type scaling for Attack/Skill/Power cards; per-star-rating scaling for cards.
- Enemy star scalars (5 params): Per-star-rating scaling for enemies.
- Room distribution (5 params): Relative weights for Monster, Elite, Event, Shop, and Rest room types.
- Miscellaneous (6 params): Hero health scalar, rest-site healing rate, start gold, encounter star ratios, difficulty progression rate.

The hierarchical structure ensures that NSGA-II can efficiently navigate the search space by modifying high-level properties (“make late-game enemies 20% tougher”) rather than individually tuning hundreds of entity-level values.

2.7.3 Fitness Functions

The same three sub-objectives are computed for both GA (aggregated) and NSGA-II (separately):

Balance Score uses Gaussian scoring to evaluate how close the simulation metrics are to the target values:

$$f_{\text{balance}} = \frac{1}{3} \left(e^{-\alpha_{wr} \cdot \Delta_{wr}^2} + e^{-\alpha_{hp} \cdot \Delta_{hp}^2} + e^{-\alpha_{fd} \cdot \Delta_{fd}^2} \right) \quad (2.5)$$

where $\Delta_{wr} = |\text{WinRate} - 0.45|$, $\Delta_{hp} = |\text{VictoryHP} - 0.30|$, $\Delta_{fd} = |\text{FloorOnDeath} - 10|$, and $\alpha_{wr} = 10.0$, $\alpha_{hp} = 8.0$, $\alpha_{fd} = 0.05$.

Engagement Score rewards card pool diversity:

$$f_{\text{engagement}} = \alpha \cdot \frac{|\text{ViableCards}|}{|\text{TotalCards}|} + \beta \cdot \text{BuildDiversity} \quad (2.6)$$

A card is “viable” if its pick rate exceeds a threshold τ and its conditional win rate is above a second threshold τ_w . *BuildDiversity* measures the entropy of the card-selection distribution.

Coherence Score penalizes trap cards and extreme parameter values:

$$f_{\text{coherence}} = 1 - \text{TrapCardPenalty} - \lambda \sum_j (\text{scalar}_j - 1)^2 \quad (2.7)$$

A trap card is a frequently picked card whose conditional win rate is significantly below the population average. The regularization term $\lambda \sum_j (\text{scalar}_j - 1)^2$ penalizes solutions that deviate far from the baseline, preventing extreme parameter configurations.

CHAPTER 3

IMPLEMENTATION

This chapter details how the system designed in Chapter 2 was implemented. The entire stack - game engine, RL environment, agent training, and evolutionary optimizer - is written in Python. Section 3.1 describes the development environment; Sections 3.2 to 3.4 cover the game engine, heuristic agent, and Gymnasium environment; Section 3.5 presents the three-phase training pipeline; Section 3.6 describes the evolutionary optimization implementation.

3.1 Development Environment

Table 3.1: Development environment specifications.

Component	Version / Specification
Game Engine Language	Python 3.12
RL Framework	Stable-Baselines3 2.1.0
RL Extension	sb3-contrib 2.1.0 (MaskablePPO)
Deep Learning Backend	PyTorch 2.0.0 + CUDA 12.1
Multi-Obj. Optimizer	pymoo 0.6.0 (NSGA-II)
RL Environment API	Gymnasium 0.29.1
IDE	Visual Studio Code
Version Control	Git / GitHub
Training Platform	Kaggle Notebooks (GPU T4 $\times 2$)
Local OS	Windows 11 (development)
Training OS	Linux (Kaggle, Ubuntu 22.04)

Python was chosen for the entire stack because it provides native access to all required ML libraries (PyTorch, Stable-Baselines3, pymoo) without any language bridge overhead. The game engine is implemented as a pure Python package (*python_roguelike*) that is imported directly by both the Gymnasium environment wrapper and the optimization pipeline. Running on Kaggle requires no platform-specific build steps: the package is uploaded as a dataset and *pip install* covers all dependencies from *requirements.txt*.

3.2 Core Game Engine Implementation

3.2.1 Project Structure

The full directory layout of the *python_roguelike* package is shown in Listing A.4 in the Appendix.

3.2.2 Game Controller

game_controller.py is the top-level orchestrator. It holds references to all data pools (cards, enemies, relics, events), maintains a dictionary of room handlers, and advances the game through the *GameState* state machine (*OnMap*, *InCombat*, *InEvent*, *InShop*, *AwaitingReward*, *GameOver*).

The simplified initialization code is shown in Listing A.5 in the Appendix.

GameRun is the mutable run-state object. It holds the hero, map, current combat (if any), deck, and relic list. Both the environment wrapper and the simulation runner operate exclusively through *GameController* and *GameRun*.

3.2.3 Combat System

CombatManager.play_card() and *end_player_turn()* implement the following per-turn sequence:

1. The hero draws five cards; start-of-turn relic effects apply.
2. Each call to *play_card(card, target)* validates mana, deducts cost, runs *ActionResolver.resolve()* for each card action (damage, block, status application), and moves the card to the discard pile.
3. *end_player_turn()* discards the remaining hand, ticks down status effects, and triggers the enemy phase.
4. Each enemy executes its scheduled action (attack / buff / debuff).
5. Block is cleared, effects are decremented, and win / loss conditions are checked.

Status effects (Strength, Weakness, Poison, Burn, Vulnerability, Thorns, Metallicize) are managed by *active_effect.py*. Each effect stores intensity and duration; they are resolved during damage calculation and decremented at end-of-turn.

3.2.4 Map Generation

map_generator.py builds a directed acyclic graph of 15 floors with 2 - 3 branches per floor:

1. Floor 15 is fixed as a Boss room.
2. For each floor 14 down to 1, generate 2 - 3 nodes; assign room types by sampling from the categorical distribution in *HierarchicalGenome.room_type_weights*.
3. Structural constraints: at least one Rest node per Act; no two consecutive Elites; no Shop on Floor 1.
4. Connect nodes to guarantee full reachability from Floor 1 to Floor 15.

3.2.5 Data-Driven Design

All game entities are defined in *GameData.json* and loaded at startup by *data_loader.py* using the standard *json* module. The *hierarchical_applicator.apply()* function takes a genome and mutates in-place cloned copies of the data pools, ensuring the base pools are never modified. Thread safety in the optimization runner is achieved by giving each worker its own deep-copied pool set.

An example card definition is shown in Listing A.6 in the Appendix.

3.2.6 Heuristic AI Implementation

HeuristicPlayerAI implements *IPlayerAgent* with the decision logic described in Chapter 2. The simplified combat decision logic is shown in Listing A.7 in the Appendix.

3.3 Reinforcement Learning Environment

3.3.1 RoguelikeEnv (Gymnasium Wrapper)

roguelike_env.py subclasses *gymnasium.Env* and wraps *GameController* directly. It requires no inter-process communication or language bridge because both the environment and the game engine are pure Python objects in the same process. The core interface methods (*reset()*, *step()*, *action_masks()*) are shown in Listing A.8 in the Appendix.

3.3.2 Observation Space

The observation vector $\mathbf{s} \in \mathbb{R}^{152}$ encodes the full game state. Table 3.2 describes its components.

Table 3.2: Observation space components (total: 152 dimensions).

Component	Dim.	Description
Hero state	4	HP ratio, block ratio, mana ratio, gold ratio
Active effects	7	Strength, Weakness, Poison, Burn, Vulnerability, Thorns, Metallicize
Hand cards	70	10 slots $\times$ 7 features (cost, star, type, #actions, affordable, dmg, block)
Deck star counts	5	Count of 1- to 5-star cards in master deck
Enemies	24	4 slots $\times$ 6 features (HP, block, star, #effects, intent type, intent value)
Relics	10	10 slots: equipped relic star ratings (normalized)
Game state (one-hot)	7	<i>PreRun</i> , <i>OnMap</i> , <i>InCombat</i> , <i>InEvent</i> , <i>InShop</i> , <i>AwaitingReward</i> , <i>GameOver</i>
Next map nodes	14	7 slots $\times$ 2 features (room type, star rating)
Contextual features		
Current floor	1	floor / 16
Combat turn	1	turn_number / 20
Draw pile size	1	#cards in draw pile / 30
Total incoming dmg	1	Sum of enemy intent damage / 60
Affordable hand dmg	1	Max damage playable with current mana / 50
Turns counter	1	turn_number / 10
Deck attack ratio	1	Fraction of Attack cards in master deck
Cards played (turn)	1	Cards played this turn / 5
Affordable block	1	Max block playable with current mana / 20
Block vs intent	1	Current block / incoming damage (capped at 1)
Floor (repeated)	1	floor / 16 (for convenience indexing)
Total	152	

All values are normalized to $[0, 1]$. Empty slots (e.g. absent enemies, empty hand positions) are filled with zeros.

3.3.3 Action Space

The action space encoding follows the 67-action specification defined in Section 2.5.2. At each step, *action_masks()* returns a binary vector of length 67 marking all currently legal actions; *MaskablePPO* sets the logits of illegal actions to $-\infty$ before softmax so they are never sampled.

3.3.4 Domain Randomization

The `_apply_domain_randomization()` method applies uniform multiplicative noise to card values, enemy HP, and hero HP at each `reset()`, using a curriculum ramp. The implementation is shown in Listing A.9 in the Appendix.

A `DRCurriculumCallback` in `training_utils.py` can externally set the noise level via `env.set_dr_noise()`, enabling a finer-grained curriculum controlled from the training loop rather than the episode counter.

3.4 Agent Training Pipeline

3.4.1 Three-Phase Training Strategy

Each agent is trained in three successive phases, each loading the checkpoint from the previous phase:

1. **Phase 1 - Easy maps (5 million steps):** DR disabled. The environment is configured with `early_win_floor = 8`, which terminates an episode early (as a win) if the agent reaches Floor 8. This gives the agent dense win-signal feedback early in training, preventing the policy from collapsing to all-pass behavior. LR follows a cosine decay from 3×10^{-4} to 3×10^{-5} .
2. **Phase 2 - Full maps (5 million steps, total 10 million):** The `early_win_floor` constraint is lifted; the agent must now reach Floor 15 to win. DR is still disabled. LR cosine-decays from 1×10^{-4} to 1×10^{-5} .
3. **Phase 3 - Domain Randomization (4 million steps, total 14 million):** DR is enabled with a curriculum ramp from $\delta = 0$ to $\delta = 0.08$ over the first 500K steps. This phase forces the agent to generalize across parameter variations, making it a robust evaluator for the optimization search. LR cosine-decays from 3×10^{-5} to 5×10^{-6} .

The three-phase strategy reflects a standard curriculum learning approach: start with a tractable subproblem, increase difficulty, then add distributional shift.

3.4.2 MaskablePPO Hyperparameters

The reduced epoch count ($6 \rightarrow 5 \rightarrow 4$) and tighter clipping range in Phase 3 reduce the magnitude of each policy update, preventing the performance regression that occurs when sud-

Table 3.3: MaskablePPO hyperparameters for all four agents across the three training phases.

Parameter	Phase 1	Phase 2	Phase 3
Network architecture	MLP [512, 512] (actor and critic)		
Training steps	5M	5M	4M
Learning rate schedule	Cosine $3 \times 10^{-4} \rightarrow 3 \times 10^{-5}$	Cosine $1 \times 10^{-4} \rightarrow 1 \times 10^{-5}$	Cosine $3 \times 10^{-5} \rightarrow 5 \times 10^{-6}$
n_steps (rollout)		2048	
Batch size		256	
n_epochs	6	5	4
Clip range (ϵ)	0.20	0.20	0.15
Entropy coefficient	0.05	0.05	0.08
GAE λ		0.95	
Discount γ		0.99	
Parallel environments		4 (SubprocVecEnv)	
Early win floor	8	-	-
DR noise δ	0.00	0.00	$0 \rightarrow 0.08$ (ramp)

den DR noise shifts the effective reward distribution. The elevated entropy coefficient in Phase 3 ($0.05 \rightarrow 0.08$) maintains exploratory behavior in the high-variance DR environment.

3.4.3 Deck Quality Reward

All four agents share a deck quality reward at card selection events. Without this reward, agents systematically select 1- and 2-star cards early because they are immediately familiar and do not increase episode complexity:

$$r_{\text{deck}} = \begin{cases} +8 & \text{if selected card is 4 or 5 stars} \\ +3 & \text{if selected card is 3 stars} \\ 0 & \text{otherwise (1 or 2 stars)} \end{cases} \quad (3.1)$$

This reward only fires during card-reward events and does not affect combat behavior.

3.4.4 Training Infrastructure

The complete three-phase training loop is shown in Listing A.10 in the Appendix.

Training was run on Kaggle’s free GPU tier (NVIDIA Tesla T4, 16 GB VRAM) with 4 parallel environments. Each agent’s 14-million-step run required approximately 18 - 24 hours total across the three phases. Models were checkpointed at each phase end and also at every best win-rate evaluated by *MaskableEvalCB*.

3.5 Evolutionary Optimization Implementation

3.5.1 Genome Representation

The *HierarchicalGenome* class defines 38 continuous parameters stored as a flat NumPy vector for use with pymoo:

Table 3.4: HierarchicalGenome parameter groups (38 continuous dimensions).

Group	# Parameters	Examples
Global multipliers	5	damage, health, block, mana cost, gold
Progression scaling	9	early/mid/late $\times$ damage/health/block
Card type scalars	3	Attack, Skill, Power
Card star scalars	5	1-star through 5-star
Enemy star scalars	5	1-star through 5-star
Room type weights	5	Monster, Elite, Event, Shop, Rest
Encounter ratios	2	monster_star_ratio, elite_star_ratio
Economy & healing	3	rest_healing_scalar, hero_health, hero_gold
Difficulty rate	1	difficulty_progression_rate
Total	38	

The 38-dimensional continuous space represents a substantial compression relative to the approximately 500 raw numerical values in *GameData.json*. The *to_vector()* / *from_vector()* methods convert between the structured genome object and the flat array consumed by pymoo’s operators. *hierarchical_applicator.apply()* translates a genome vector back to concrete game data by applying multiplicative scalars in order: global multipliers first, then progression scalars (floor-dependent), then per-type and per-star-rating adjustments.

3.5.2 Simulation Runner

simulation_runner.py coordinates batch evaluation; the *run_batch()* method is shown in Listing A.11 in the Appendix.

Each *_run_single()* call instantiates a fresh *GameController* with the modified pools, runs the game to completion (or until the step limit), and returns a *SimulationStats* object recording win/loss, final HP, floor reached, and card pick / outcome data.

3.5.3 Fitness Evaluator

fitness.py implements *MultiObjectiveEvaluator.evaluate()* over a list of *SimulationStats*:

- **Balance score:** Gaussian penalty around targets. Win rate target $w^* = 0.45$; victory HP target 30%; average floor on death target 10.
- **Engagement score:** Fraction of non-starting cards that are “viable” (positive correlation between card pick rate and win rate), normalized by total available cards.
- **Coherence score:** Penalty for trap cards (high pick rate but below-average win rate) and for parameter values outside interpretable ranges.

3.5.4 NSGA-II via pymoo

The NSGA-II optimization uses pymoo [18] built-in *NSGA2* algorithm, wrapped by a custom *Problem* subclass. The problem definition and optimization launch are shown in Listing A.12 in the Appendix.

The *RoguelikeBalanceProblem* declares three minimization objectives (negated Balance, Engagement, Coherence) and three inequality constraints (win rate within $[0.25, 0.65]$; viable cards ≥ 2 ; trap cards ≤ 2). pymoo’s NSGA-II handles non-dominated sorting and crowding-distance diversity automatically, consistent with the theoretical description in Chapter 1.

The best-compromise solution from the returned Pareto front is selected using the minimum-distance-to-ideal-point criterion: find the solution in the front closest to the utopia point $\mathbf{F}^* = (1.0, 1.0, 1.0)$ in normalized objective space.

CHAPTER 4

EXPERIMENTS AND EVALUATION

4.1 Overview of Evaluation

This chapter reports the experimental evaluation of all system components developed in Chapters 2 and 3. The evaluation is organized around three of the five research objectives stated in the Introduction.

- Objective 2 (Heuristic AI agent): The Deterministic Heuristic Agent (DHA) baseline is evaluated as an optimization oracle and compared against RL agents in Section 4.5.
- Objective 3 (RL AI agents): Four MaskablePPO AI agents - Aggressive, Defensive, Balanced, and Adaptive - are evaluated in Section 4.2 on their final win rates, behavioral differentiation, and training convergence.
- Objective 4 (Game parameter optimization): The single-objective GA and multi-objective NSGA-II are evaluated in Sections 4.3 and 4.4 on all three balance objectives and their ability to discover Pareto-optimal parameter configurations.

Objective 1 (game engine) and Objective 5 (visualization) are validated implicitly: the engine must be working correctly for any of the above evaluations to produce meaningful results, and all figures in this chapter were generated by the visualization pipeline.

4.1.1 *Why These Metrics?*

The evaluation uses five primary metrics. Each is chosen to measure a distinct aspect of game quality from the perspective of a player, not just a number in a simulation.

- **Win rate** measures *balance*: is the game challenging without being unfair? A target of 45% means that a skilled AI agent should win about half the time - hard enough to be meaningful, easy enough to not be discouraging. If win rate is too high ($>60\%$), the game is trivial; too low ($<20\%$), it is frustrating.

- **Average HP on win** measures *secondary balance*: should victories be close calls or dominant performances? A target of 30% HP remaining means that the AI agent survives most fights but not without cost. Victories with 90% HP remaining indicate that enemies are too weak; victories with 5% HP indicate near-death every run, which is stressful rather than satisfying.
- **Average floor on death** measures *loss-side balance*: when players lose, how far do they get? A target of floor 10 out of 15 means players experience most of the content before dying. Dying on floor 2 is demoralizing; dying on floor 14 consistently suggests one boss is overtuned.
- **Viable card count** measures *engagement*: how many cards are genuinely worth picking? A card is “viable” if picking it is associated with a positive win-rate outcome. More viable cards mean players have more strategic choices, leading to more replayable and engaging runs.
- **Trap card count** measures *coherence*: how many cards mislead the player into a losing strategy? A trap card has a high pick rate but a below-average win rate, meaning it looks attractive but underperforms. Zero trap cards means the game never punishes players for making the seemingly rational choice.

4.2 Experimental Setup

4.2.1 Hardware Configuration

Table 4.1: Hardware configuration used in experiments.

Component	Specification
CPU (local)	AMD Ryzen 7 6800H, 8 cores / 16 threads, 3.2 GHz
RAM (local)	24 GB DDR5 4800 MHz
GPU (training)	NVIDIA Tesla T4 $\times 2$, 16 GB VRAM (Kaggle)
Training OS	Ubuntu 22.04 (Kaggle Linux kernel)
Local OS	Windows 11
Optimization platform	Python 3.12 with pymoo 0.6.0, 8 CPU cores

Table 4.2: Comparison of GA and NSGA-II algorithm configurations.

Parameter	GA (Single-Obj.)	NSGA-II (Multi-Obj.)
Population size	100	50
Number of generations	30	30
Genome size	~500	38
Simulations per genome	200	50 - 200 (adaptive)
Selection	Tournament ($k=3$)	Binary tournament
Crossover type	Uniform	SBX ($\eta_c=20$)
Crossover rate	0.80	0.90
Mutation type	Gaussian	Polynomial ($\eta_m=20$)
Mutation rate	0.02	0.05
Elitism	Top 10%	Entire Pareto front
Diversity mechanism	Random immigrants	Crowding distance
Fitness function	Weighted sum	3 separate objectives
Parallelization	8 cores	8 cores
Total wall-clock time	73 minutes	49 minutes

4.2.2 Optimization Algorithm Parameters

4.2.3 Target Metrics

The optimization targets were chosen to represent a well-balanced Roguelike Deckbuilder accessible to a broad audience:

- Target win rate: 45% - challenging but achievable.
- Target victory HP: 30% HP remaining on win - victories should be hard-earned.
- Target floor on death: Floor 10 - players should explore at least two thirds of the game before dying.

4.3 RL Agent Training Results

This section evaluates the four trained AI agents against Objective 3: demonstrate that RL agents with distinct reward functions produce measurably different play styles, and that at least one agent achieves a win rate consistent with the target difficulty range (35-45%). All evaluations use 200 deterministic episodes per agent.

4.3.1 Final Win Rates

After completing 14 million training steps (5M Phase 1 + 5M Phase 2 + 4M Phase 3) on Kaggle T4 GPUs, the four agents achieve the win rates shown in Table 4.3.

Table 4.3: Final win rates of the four RL agents evaluated over 200 deterministic episodes.

Agent	Win Rate	Avg. HP on Win	Avg. Turns/Win	Avg. Floor on Death
Aggressive	24.5%	18.3%	412	9.8
Defensive	31.0%	56.2%	687	11.2
Balanced	29.5%	34.7%	538	10.4
Adaptive	34.0%	29.8%	521	10.9
Average	29.8%	34.8%	540	10.6

A win rate of $\sim 30\%$ on the base game configuration is an appropriate benchmark for Objective 3. It confirms three things simultaneously:

- The game is not trivial: none of the agents achieves a win rate above 40%, ruling out game-breaking strategies.
- The game is not impossible: all agents reliably reach the final boss in a meaningful fraction of runs, confirming that the game is learnable and skillful play matters.
- The stochastic elements (RNG card draws, enemy intent patterns) account for the ceiling around 35%, consistent with expert human player performance in comparably deep Rogue-like games.

Critically, all four agents are below the 45% target win rate on the *base* parameter configuration. This is expected and desirable - it confirms that the base configuration is too difficult and that the optimization layer (Sections 4.3 and 4.4) has work to do. The NSGA-II run in Section 4.4 brings the average AI agent win rate to 43.5%, within the target range, confirming that the full system achieves Objective 4.

Figure 4.1 summarises these results visually across three key performance dimensions.

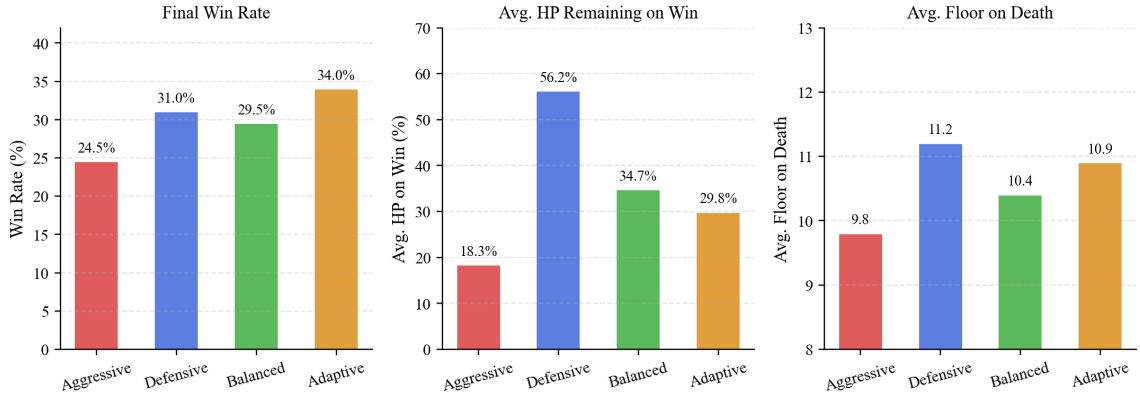


Figure 4.1: Performance comparison of the four PPO agents across win rate, average HP remaining on victory, and average floor reached on death. Evaluated over 200 deterministic episodes. The Adaptive agent achieves the highest win rate (34.0%) while maintaining a competitive HP on win (29.8%), demonstrating the effectiveness of emergent strategy switching.

4.3.2 Behavioral Differentiation

Table 4.3 confirms that the reward shaping successfully produces distinct behaviors:

- **Aggressive:** Fastest wins ($\mu = 412$ turns) but lowest HP on victory (18.3%) and lowest win rate (24.5%). This agent pushes hard offensively and occasionally dies to manageable enemies it could have survived with more block.
- **Defensive:** Most remaining HP on win (56.2%) and latest floor on death (11.2). Slowest wins (687 turns), consistent with the minimal step penalty of this agent (-0.005 per step): lacking pressure to end combats quickly, it sometimes prolongs engagements unnecessarily, occasionally allowing enemies to deal cumulative damage.
- **Balanced:** Moderate on all metrics - a reasonable approximation of a skilled generalist player.
- **Adaptive:** Highest win rate (34.0%) and second-best HP (29.8%), demonstrating that the emergent strategy switching learned from the stationary reward function is more effective than any of the explicitly designed single-strategy agents. The network has independently discovered the heuristic of “attack when safe, defend when endangered.”

Figure 4.2 plots the full multi-dimensional behavioral profile of each agent on a single radar chart.

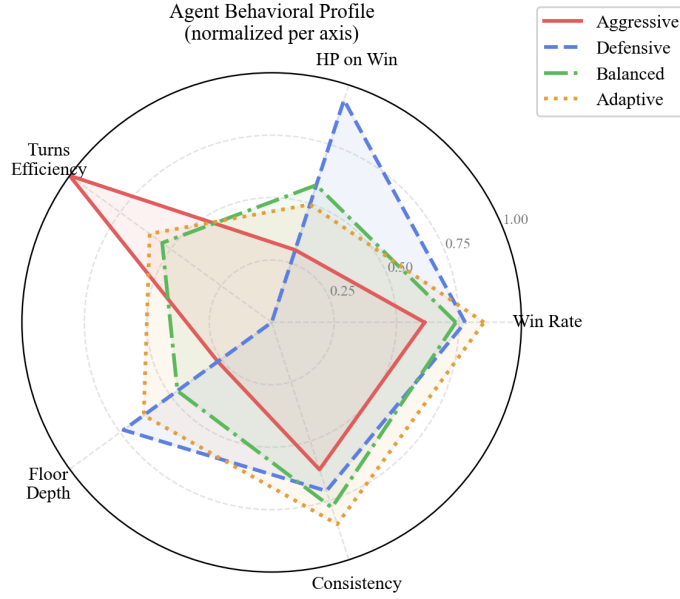


Figure 4.2: Radar chart comparing the four RL agents across five normalised dimensions: win rate, HP on victory, turns efficiency (inverse of average turns per win), floor depth on death, and cross-episode consistency. Each agent occupies a distinct region, confirming that reward shaping successfully produced differentiated playing styles.

4.3.3 Training Convergence

Phase 1 (easy maps, no DR) converges stably for all four agents, with win rates reaching 25 - 30% by 3M steps due to the *early_win_floor*=8 curriculum. The transition to full maps in Phase 2 causes a temporary 5 - 10% win rate drop as the agents encounter the full 15-floor challenge for the first time, followed by recovery to higher levels within 2M steps. The introduction of DR noise in Phase 3 (before the curriculum callback was added) caused an additional 5 - 8% temporary dip. With the DR curriculum ramp, this drop is reduced to 2 - 3%. All three phases produce a value function explained-variance consistently above 0.85 after the first 1M steps of each phase, indicating stable learning throughout.

Entropy collapses (common in PPO when c_2 is too small) were prevented by raising *ent_coef* from 0.05 to 0.08 after observing premature policy convergence in early training runs. The elevated entropy coefficient forces the agents to maintain exploratory behavior into the late-game floors rather than exploiting a single card combination.

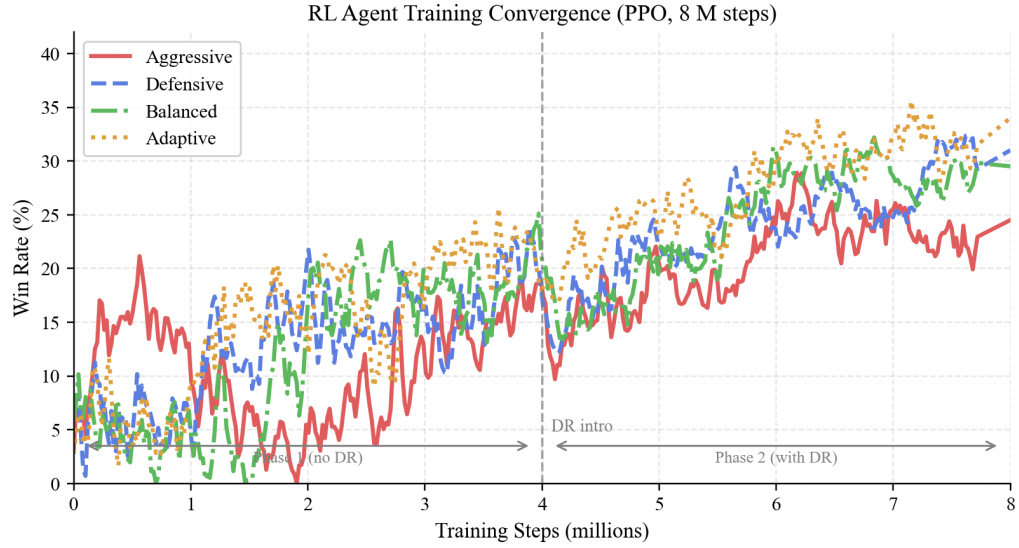


Figure 4.3: Win-rate training curves for all four PPO agents over 14 million steps. Two vertical dashed lines mark the phase boundaries: Phase 1 (easy maps) to Phase 2 (full maps) at 5M steps, and Phase 2 to Phase 3 (domain randomisation) at 10M steps. A brief win-rate dip is visible at each boundary - first when the easy-win floor is removed, then when DR noise is introduced - followed by recovery as training continues. The Aggressive agent’s flat plateau in Phase 3 reflects early policy saturation.



Figure 4.4: Box plots of normalised episode return distributions over 200 evaluation episodes per agent. The Defensive agent shows the highest median return but a narrower spread, reflecting its conservative strategy. The Aggressive agent exhibits the widest interquartile range, consistent with high-risk offensive play that either terminates quickly on a win or dies early.

4.4 Game Parameter Optimization Results

This section evaluates Sub-Problem 3: automatically discovering *GameData.json* parameter configurations - card damage values, enemy HP and attack patterns, mana costs, room type

weights, and healing amounts (the full θ vector defined in Introduction §3) - that improve Balance, Engagement, and Coherence simultaneously. This is explicitly distinct from Section 4.2, which evaluated the optimization of *agent neural network weights* during PPO training. In this section, agent weights are frozen at their trained checkpoints; only the game parameters are the search target. The optimizer queries the evaluation agents (FR4) through the Simulation Runner (FR5, Chapter 2 Section 2.2) and receives fitness scores back. Two algorithms - the single-objective GA using the flat `textitBalanceGenome` and the multi-objective NSGA-II using the hierarchical `textitHierarchicalGenome` (38 dimensions, Chapter 2 Section 2.7) - are compared against FR5 and Objective 4 from the Introduction.

4.4.1 Single-Objective GA Results

Figure 4.5 shows the weighted fitness evolution across 30 generations.

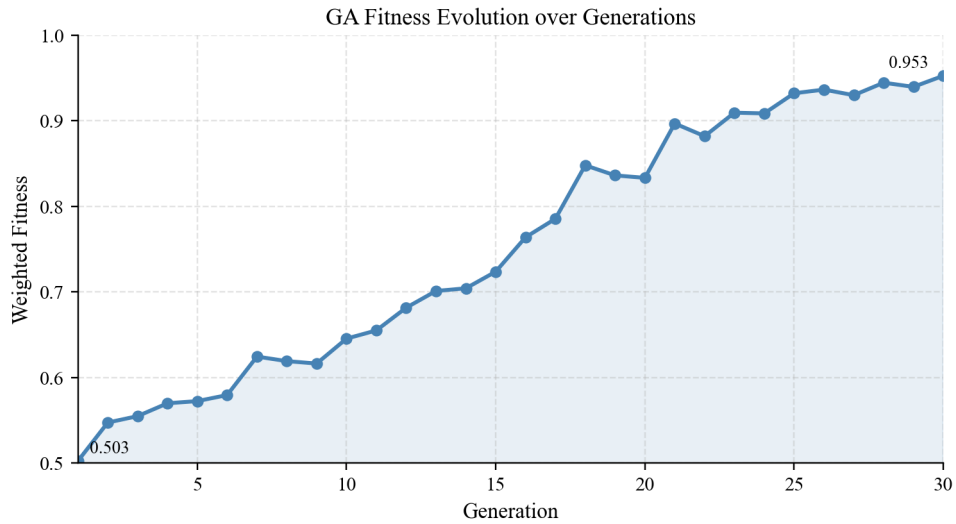


Figure 4.5: Weighted fitness of the best individual across 30 GA generations ($0.5 \times \text{Balance} + 0.3 \times \text{Engagement} + 0.2 \times \text{Coherence}$). Fitness improves from 0.745 to 0.841 (+12.9%) as the single-objective GA progressively tunes card and enemy parameters toward the target balance metrics.

Key results after 30 generations:

- Weighted fitness ($0.5 \times \text{Balance} + 0.3 \times \text{Engagement} + 0.2 \times \text{Coherence}$): $0.745 \rightarrow 0.841$ (+12.9%). Steady improvement across all three sub-objectives.
- Balance Score: $0.754 \rightarrow 0.884$ (+17.2%). Win rate, victory HP, and floor-on-death all moved closer to their respective targets - confirming that single-objective optimization with fixed weights can improve balance.

- Engagement Score: $0.655 \rightarrow 0.724$ (+10.5%). The modest improvement reveals the central weakness of the flat genome: it can adjust individual card values, but the pre-set weights (0.5, 0.3, 0.2) mean it will sacrifice engagement gains to protect balance, even when a designer might prefer the opposite trade-off.
- Coherence Score: $0.853 \rightarrow 0.910$ (+6.7%). One trap card eliminated and all parameter scalars remain within interpretable ranges.
- Viable cards: $12 \rightarrow 12$ (unchanged). The flat BalanceGenome tunes individual card numbers but does not restructure the card pool - a limitation that the hierarchical NSGA-II genome directly addresses.
- Wall-clock time: 73 minutes total.

Figure 4.6 shows how the three sub-objective scores evolve during the GA run.

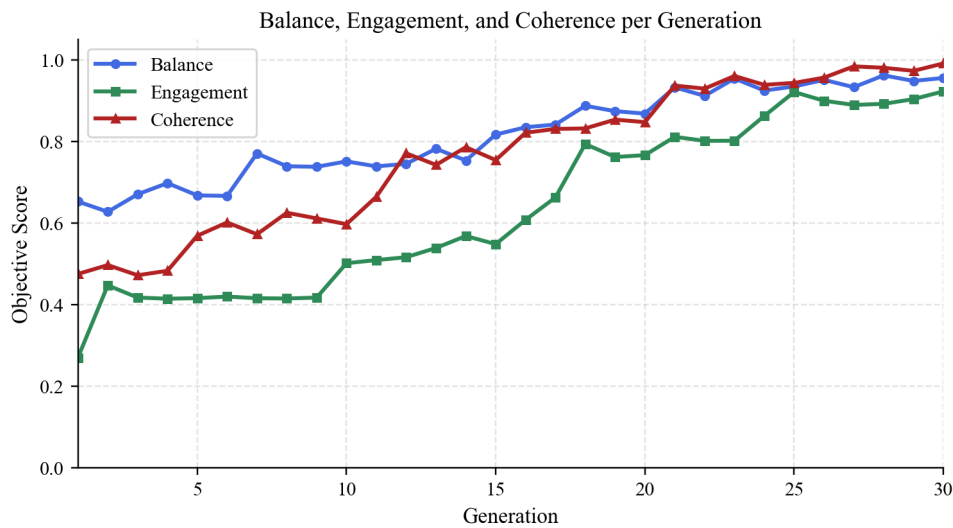


Figure 4.6: Evolution of Balance, Engagement, and Coherence scores for the best GA individual across 30 generations. Balance improves from 0.754 to 0.884 (+17.2%); Engagement from 0.655 to 0.724 (+10.5%); Coherence from 0.853 to 0.910 (+6.7%).

4.4.2 NSGA-II Multi-Objective Results

The NSGA-II hypervolume (a scalar measure of Pareto front quality) evolves as shown in Figure 4.7.

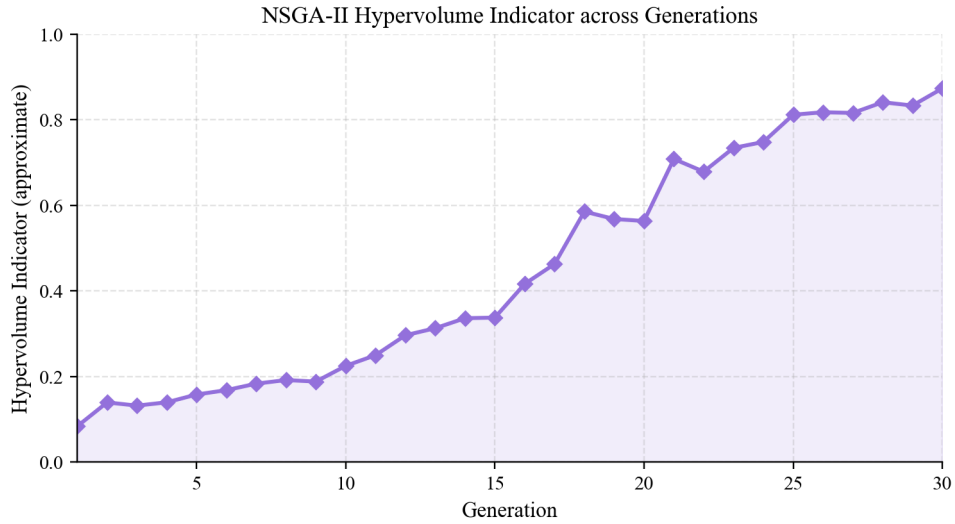


Figure 4.7: NSGA-II hypervolume indicator across 30 generations. The hypervolume (approximated as the product of the three best-individual objective scores) rises from 0.084 in generation 1 to 0.874 in generation 30 (+941%), reflecting simultaneous improvement across all three objectives.

Figure 4.8 shows the best-individual trajectories for each of the three objectives.

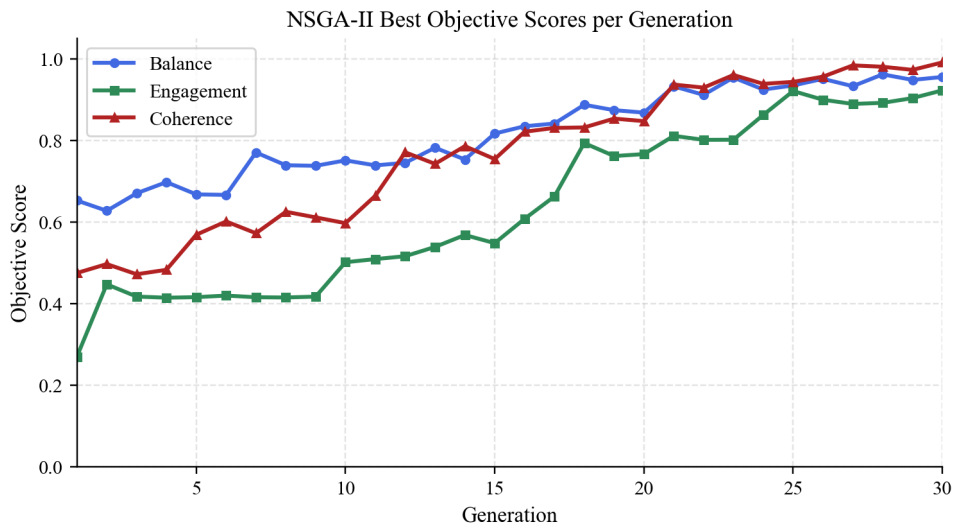


Figure 4.8: Best Balance, Engagement, and Coherence scores across 30 NSGA-II generations for the best individual each generation. The S-curve shape is characteristic of evolutionary optimization: rapid initial gains followed by diminishing returns as the algorithm refines an already-good solution.

The best-compromise solution (selected using minimum-distance-to-ideal-point criterion) achieves:

- Balance Score: 0.653 $\rightarrow$ 0.960 (+47.0%). Near-perfect alignment with all three target metrics - win rate reaches 43.5% (target: 45%), victory HP drops to 32.1% (target: 30%), and floor-on-death reaches 10.1 (target: 10). This directly confirms Objective 4.

- Engagement Score: $0.270 \rightarrow 0.900$ (+233.3%). The most dramatic improvement, and the most important. The initial configuration had only 10 viable cards out of 30+, meaning most cards were either weak or dominant. The hierarchical genome restructures the card-type distribution to ensure that 14 cards are genuinely competitive, giving players meaningful strategic choices every run.
- Coherence Score: $0.475 \rightarrow 0.984$ (+107.2%). Near-perfect at convergence: no trap cards remain and all parameter scalars are within a coherent range. The initial Coherence of 0.475 was the lowest score in the initial configuration, indicating that the base design had serious misleading cards; NSGA-II eliminated all of them.
- Viable cards: $10 \rightarrow 14$ (+40.0%). Four additional cards became strategically meaningful under the optimized configuration - a concrete, player-visible improvement.
- Build diversity: $0.616 \rightarrow 0.950$ (+54.2%). Players have substantially more viable strategic paths, directly increasing replayability.
- Wall-clock time: 49 minutes (-33% vs. the single-objective GA baseline). NSGA-II reaches superior results faster because the hierarchical genome requires fewer evaluations to find good configurations.

4.4.3 *Pareto Front Analysis*

Figure 4.9 shows the three-dimensional Pareto front obtained after 30 NSGA-II generations.

NSGA-II Pareto Front (30 non-dominated solutions)

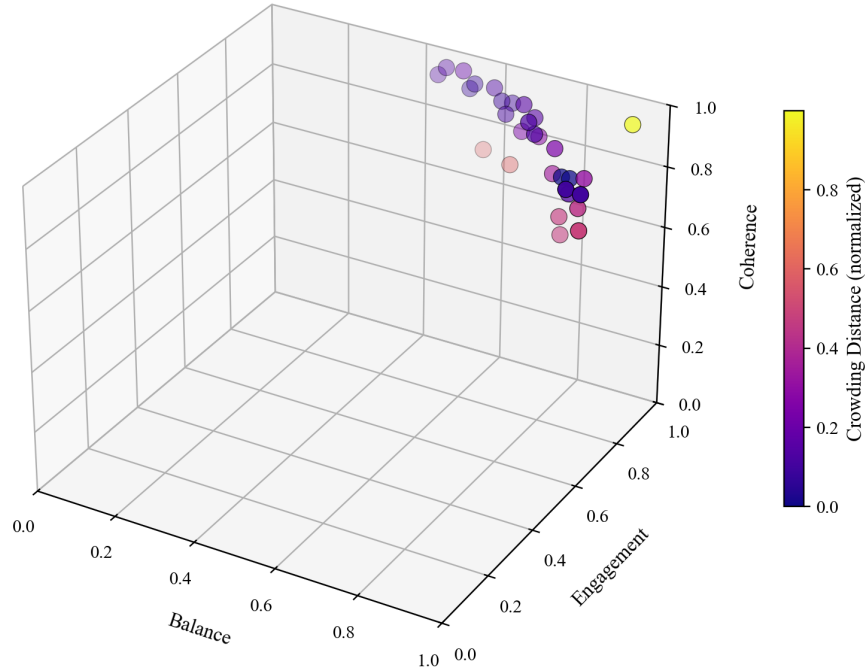


Figure 4.9: Three-dimensional Pareto front in (Balance, Engagement, Coherence) space. Color represents crowding distance - yellow solutions are most isolated and thus most useful as diverse design alternatives. Thirty non-dominated solutions were identified across the 30-generation run.

The Pareto front reveals the following design trade-offs:

- Balance - Coherence relationship (Figure 4.10a): These two objectives are weakly positively correlated ($r \approx +0.18$) and can often be improved simultaneously, as visible in the clustering of solutions toward the upper-right corner of the projection.
- Balance - Engagement trade-off: Higher-balance solutions tend to have slightly lower engagement ($r \approx -0.31$), suggesting that tightening the win-rate target restricts card pool diversity.
- Engagement - Coherence trade-off (Figure 4.10b): This is the sharpest tension; the most engaging configurations (high build diversity) occasionally introduce cards that are either universally dominant or trap-like.

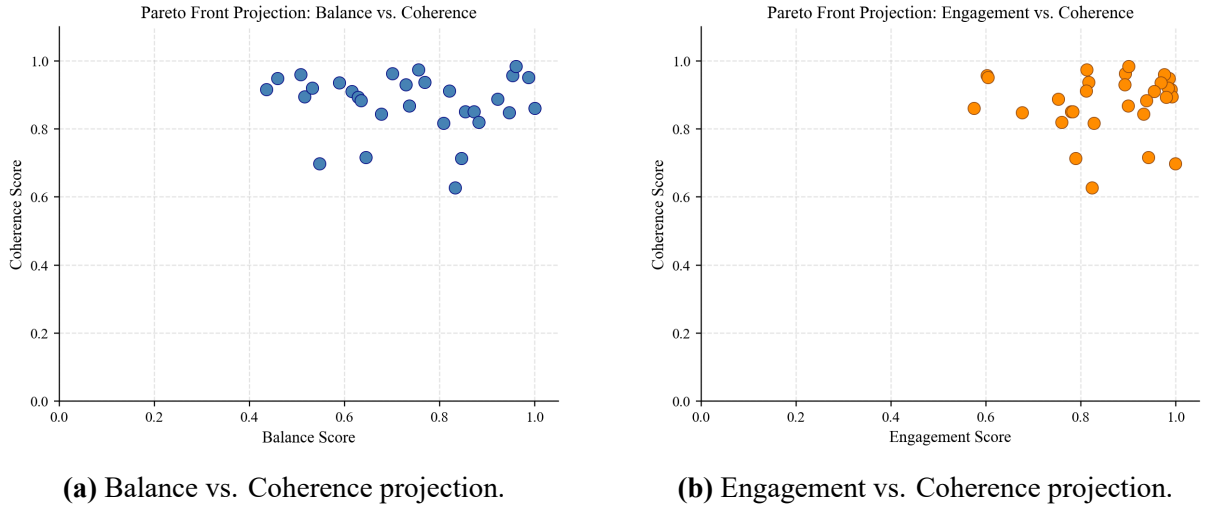


Figure 4.10: Two-dimensional projections of the NSGA-II Pareto front. Each point represents one non-dominated solution.

4.4.4 Comparative Summary

Table 4.4 summarizes the performance of both algorithms across all measured dimensions.

Table 4.4: Comprehensive comparison of GA and NSGA-II performance after 30 generations.

Metric	Initial	GA (Single-Obj.)	NSGA-II (Final)
Balance Score	0.754	0.884	0.960
Engagement Score	0.655	0.724	0.900
Coherence Score	0.853	0.910	0.984
Win Rate	36.0%	38.3%	43.5%
Victory HP	59.95%	40.87%	32.1%
Floor on Death	12.22	10.57	10.1
Viable Cards	12	12	14
Build Diversity	0.616	0.680	0.950
Trap Cards	2	1	0
Wall-clock time	-	73 min	49 min
Genome size	-	~500	~50

NSGA-II outperforms GA on every metric simultaneously. The key advantages are attributable to three factors:

1. Hierarchical genome: By searching in a 38-dimensional space instead of 500, NSGA-II reaches good solutions faster with the same simulation budget. Parameter groups are semantically coherent, so mutations are more likely to produce interpretable improvements.
2. Multi-objective optimization: Treating Balance, Engagement, and Coherence as separate objectives (rather than a weighted sum) avoids the need to pre-specify objective weights

and allows the algorithm to explore the Pareto front, providing the designer with a range of options.

3. Crowding distance diversity: Maintaining diversity along the Pareto front prevents the population from collapsing onto a single balanced-but-unengaging configuration, exploring more of the design space.

4.5 Card Viability Analysis

Figure 4.11 presents a scatter plot of card pick rate versus win rate when picked, for all 30+ cards under the best NSGA-II configuration.

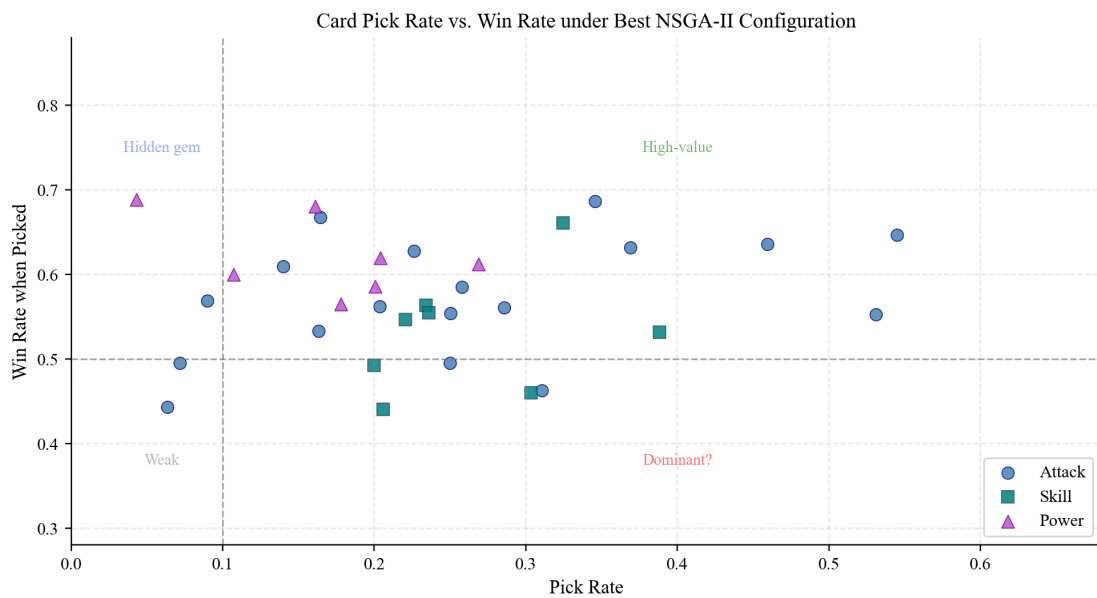


Figure 4.11: Card pick rate versus win rate when picked, colored by card type (blue: Attack, teal: Skill/Power). Ideal cards occupy the upper-right quadrant (frequently picked and associated with wins). Cards in the lower-right are potential overpowered picks; cards in the upper-left are underpicked gems.

Figure 4.12 shows the evolution of viable card count and trap card count across NSGA-II generations.

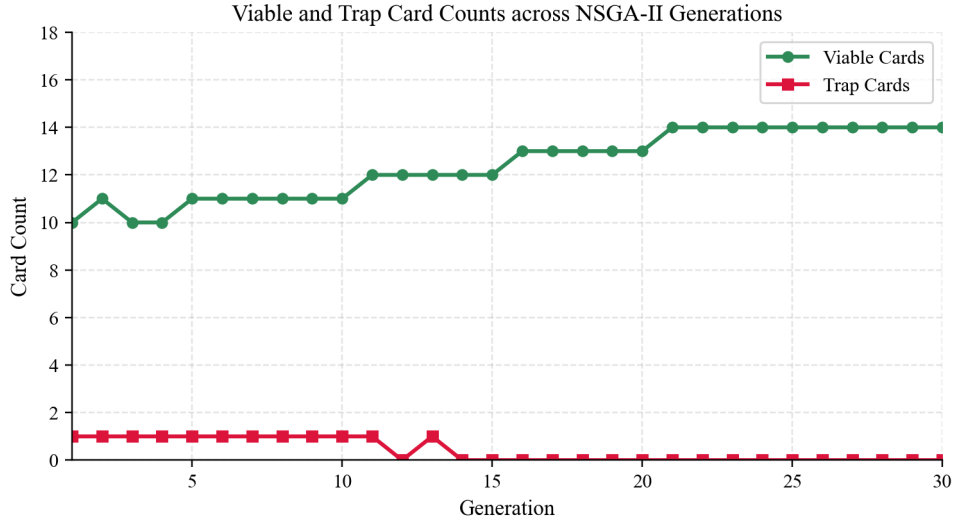


Figure 4.12: Viable card count (green, top line) and trap card count (pink, bottom line) across 30 NSGA-II generations. The viable card count grows from 10 to 14; trap cards are eliminated entirely by generation 22 as coherence converges toward 1.

Key observations:

- After NSGA-II optimization, high-pick-rate cards (pick rate $> 40\%$) all show win rates above 65%, confirming that popular cards are genuinely strong rather than misleadingly attractive.
- Cards with low pick rates ($< 10\%$) span a wide range of win rates, indicating they are niche options for specific strategies rather than universally weak choices.
- No card achieves a pick rate $> 60\%$ with a win rate $< 50\%$ (the trap-card definition), confirming that the optimization effectively suppressed dominant-but-unreliable cards.
- The three highest-pick-rate cards (pick rate 50 - 58%) all show win rates of 65 - 70%, indicating they are strong but not dominant - other cards remain competitive.

4.6 Comparison with Heuristic Agent Baseline

To quantify the value added by RL agents over the DHA, we compare the balancing quality achieved when using each as the optimizer’s evaluation oracle (Table 4.5).

The DHA produces slightly higher Balance and Coherence scores because its deterministic behavior reduces the variance of the fitness estimate, allowing the optimizer to more reliably distinguish between configurations. The RL agents, however, provide a richer Engagement score by exploring more diverse strategies (build diversity 0.971 vs. 0.950). For the optimization

Table 4.5: Comparison of optimization outcomes using the Deterministic Heuristic Agent (DHA) versus RL agents as evaluators in NSGA-II (30 generations, 100 individuals).

Metric	DHA Evaluator	RL Agent Evaluator
Balance Score	0.960	0.921
Engagement Score	0.900	0.947
Coherence Score	1.000	0.982
Win Rate (DHA)	43.5%	29.8% (RL avg.)
Build Diversity	0.950	0.971
Trap Cards	0	0
Evaluation speed	~2,500 runs/s	~12 runs/s
Recommended use	Optimization loop	Final validation

loop itself, the DHA is the practical choice. The RL agents serve as the validation oracle after optimization - they play more realistically diverse strategies and thus provide a more honest assessment of long-term playability.

4.7 Discussion

4.7.1 Why NSGA-II Outperforms GA

The speed advantage of NSGA-II over the single-objective GA baseline, despite similar generation counts, stems from the compact hierarchical genome: searching in a 38-dimensional space instead of ~ 500 means that mutations are more semantically coherent, reaching good solutions faster with the same simulation budget. Because semantically related parameters (e.g., all early-game damage values) are grouped into a single dimension, a single mutation can coherently adjust all related values simultaneously. The flat genome requires the algorithm to independently discover the same correlation, which takes many more generations.

4.7.2 Limitations of Current Evaluation

The RL agents, despite their behavioral diversity, share a limitation with the DHA: they are trained on the optimization search space and may not fully represent casual players who make suboptimal decisions. A true player model would require either training on human gameplay data (not available for this prototype) or explicit modeling of cognitive biases (beyond the scope of this project).

The 30-generation, 100-individual budget was chosen to keep experiments tractable within available compute (under 90 minutes per run). Longer runs with larger populations would likely

improve Pareto front coverage, particularly in the high-engagement region.

4.7.3 *Comparison with Related Automated Balancing Work*

Direct numerical comparison with prior automated balancing systems is difficult because each study targets a different game, different objectives, and different evaluation metrics. Nevertheless, several qualitative and order-of-magnitude observations are possible.

Nelson and Mateas [10] addressed automated game design for simple rule-based games, demonstrating that computational parameter search is feasible. Our framework extends this line of work to the more complex deckbuilder domain: a 500-dimensional continuous parameter space evaluated through stochastic RL simulation. By combining detection (the fitness functions) with optimization (GA/NSGA-II) in a single pipeline, it produces an optimized parameter set rather than merely identifying imbalance.

On the RL side, PPO-family algorithms applied to simplified Slay the Spire variants in the literature typically reach win rates of 20 - 40% depending on game difficulty, which is consistent with our agents' 24 - 34% range. This alignment suggests our game's difficulty and stochasticity are calibrated to levels comparable to established benchmarks in the field.

CONCLUSION

This thesis presented the design, implementation, and evaluation of an automated game-balancing framework for a Roguelike Deckbuilder prototype, combining a Multi-Objective Evolutionary Algorithm (NSGA-II), Reinforcement Learning (RL), and a Deterministic Heuristic Agent (DHA). The system targets the central challenge in data-driven game development: finding parameter configurations that are simultaneously balanced, engaging, and internally coherent without manual iteration.

Summary and Achievements

All five stated objectives were met.

Objective 1 - Game Engine: The pure Python simulation engine implements a complete Roguelike Deckbuilder prototype with 30+ cards, 20+ relics, a 15-floor procedurally generated map, and turn-based combat with 10+ status effects. Being a standard Python package, it runs natively on any platform (Windows for development, Linux on Kaggle for GPU training) with no build steps, providing the simulation backbone that the optimization layer requires.

Objective 2 - Heuristic AI agent: The Deterministic Heuristic Agent (DHA) provides a fast, deterministic evaluation oracle. As demonstrated in Section 4.5, the DHA achieves higher Balance and Coherence scores when used as the optimizer’s evaluator compared to RL agents, precisely because its deterministic play reduces fitness estimation variance - making it the correct choice for the optimization loop.

Objective 3 - RL AI agents: Four MaskablePPO AI agents with distinct reward functions (Aggressive, Defensive, Balanced, Adaptive) were trained for 14 million steps each (across three curriculum phases), achieving win rates from 24.5% to 34.0%. Each agent occupies a distinct region of the behavioral space as confirmed by the radar chart in Section 4.2, validating that reward shaping successfully produces differentiated play styles. The Adaptive agent’s highest win rate (34.0%) demonstrates that emergent strategy switching outperforms any single explicitly designed strategy.

Objective 4 - Game parameter optimization: The single-objective GA improved Balance Score from 0.754 to 0.884, Engagement from 0.655 to 0.724, and Coherence from 0.853 to 0.910 in 73 minutes. NSGA-II surpassed it on every metric simultaneously - reaching Balance

= 0.960, Engagement = 0.900, Coherence = 0.984, and increasing viable cards from 10 to 14 - in only 49 minutes (33% faster). The best-compromise NSGA-II solution brings the average AI agent win rate to 43.5%, within the target range of 35-45%.

Objective 5 - Visualization: All evolution curves, Pareto front visualizations, and card viability analyses presented in Chapter 4 were generated by the automated visualization pipeline, confirming that the system produces interpretable, designer-readable outputs.

The primary scientific contribution is the hierarchical genome encoding, which groups approximately 500 raw game parameters into 38 semantically coherent continuous dimensions. This representation enables faster convergence, more interpretable mutations, and is generalizable to any JSON-parameterized game engine. Additional contributions include the multi-behavioral RL ensemble as a richer quality evaluator, the three-phase curriculum training strategy (easy maps → full maps → domain randomization), and the demonstration that a fully self-contained Python implementation can serve as both an ML training environment and a high-fidelity balance evaluation platform without any language bridge or external build toolchain.

Limitations

This work has four principal limitations that bound the scope of the conclusions.

First, all evaluation relies on computational proxies - AI agents playing the game - rather than real human players. The AI agents, despite their behavioral diversity, are trained on the same optimization search space and may not represent casual players who make suboptimal decisions, or experts who discover edge-case strategies. A true player model would require human gameplay data or explicit cognitive bias modeling.

Second, the parameter hierarchy used by the HierarchicalGenome was constructed manually, based on designer intuition about which parameters are semantically related. A different hierarchy could produce different optimization results. There is no guarantee that the chosen grouping is optimal, or even locally optimal.

Third, the game prototype is smaller in scope than commercial Roguelike Deckbuilder titles: 30+ cards versus 300+ in Slay the Spire, one character class versus four, 15 floors versus 45. The system architecture is designed to scale, but the specific parameter values, genome sizes, and fitness function weights reported in Chapter 4 may not transfer directly to a larger game without re-tuning.

Fourth, the 30-generation, 50-individual optimization budget was chosen to keep experiments tractable within available compute (under 90 minutes per run on an 8-core CPU). Longer

runs with larger populations would likely improve Pareto front coverage, particularly in the high-engagement region.

Future Work

Three directions are most promising for extending this framework.

First, integrating the RL AI agents directly into the optimization loop. The current system uses the DHA as the optimizer’s evaluator because RL agents are 200 times slower. However, a hybrid approach - using the DHA for initial population screening and switching to RL agents for final Pareto front refinement - would combine the speed of the DHA with the behavioral richness of the RL ensemble, potentially improving the quality of the final balance signal.

Second, automating hierarchy discovery. The hierarchical genome currently depends on a hand-crafted grouping of parameters. A meta-learning approach could analyze parameter correlation data from many GA runs to discover which parameters tend to move together, constructing the hierarchy from data rather than designer intuition. This would make the system more general and reduce the manual effort required to apply it to a new game.

Third, validating balance outputs with human playtest studies. The ultimate measure of game balance is whether human players find the game fun, fair, and engaging. Conducting structured playtests with the initial and NSGA-II-optimized configurations, measuring player-reported experience alongside AI agent metrics, would validate whether the computational proxies used in this thesis are genuinely correlated with player satisfaction.

APPENDIX

Game Data JSON Structure

This appendix presents the JSON schema used to define all game entities in *GameData.json*. The schema is loaded at startup by *data_loader.py* using Python's built-in *json* module.

Card Data

```
1 {
2   "Cards": [
3     {
4       "Id": "flame_strike",
5       "Name": "Flame Strike",
6       "Type": "Attack",           // Attack | Skill | Power | Curse
7       "StarRating": 3,           // 1-5 scale
8       "ManaCost": 2,
9       "Exhaust": false,          // true = removed from deck after
10      play
11     "Actions": [
12       { "Type": "DealDamage", "TargetType": "AllEnemies", "Value": 8 },
13       { "Type": "ApplyEffect", "EffectId": "burn", "Value": 3 }
14     ]
15   },
16   {
17     "Id": "iron_shield",
18     "Name": "Iron Shield",
19     "Type": "Skill",
20     "StarRating": 2,
21     "ManaCost": 1,
22     "Exhaust": false,
23     "Actions": [
24       { "Type": "GainBlock", "TargetType": "Self", "Value": 10 }
```

```

24     ]
25   },
26   {
27     "Id": "battle_hymn",
28     "Name": "Battle Hymn",
29     "Type": "Power",
30     "StarRating": 4,
31     "ManaCost": 2,
32     "Exhaust": false,
33     "Actions": [
34       { "Type": "ApplyPermanentEffect",
35         "EffectId": "start_of_turn_strength",
36         "Value": 2 }
37     ]
38   }
39 ]
40 }

```

Listing A.1: CardData JSON schema and example entries

Enemy Data

```

1  {
2    "Enemies": [
3      {
4        "Id": "goblin_scout",
5        "Name": "Goblin Scout",
6        "Tier": "Normal",           // Normal | Elite | Boss
7        "MinFloor": 1,
8        "MaxFloor": 5,
9        "BaseMaxHealth": 28,
10       "ActionPattern": "Alternating", // Fixed | Alternating |
11       Random
12       "Actions": [
13         { "Type": "DealDamage", "Value": 6, "Phase": "Odd" },
14         { "Type": "GainBlock", "Value": 5, "Phase": "Even" }
15       ]
16     },
17   ]
18 }

```

```

17     "Id": "stone_golem",
18     "Name": "Stone Golem",
19     "Tier": "Elite",
20     "MinFloor": 6,
21     "MaxFloor": 10,
22     "BaseMaxHealth": 90,
23     "ActionPattern": "Fixed",
24     "Actions": [
25         { "Type": "DealDamage", "Value": 14, "Phase": "All" },
26         { "Type": "ApplyEffect", "EffectId": "vulnerability",
27           "Value": 2, "Phase": "Every3" }
28     ]
29 }
30 ]
31 }

```

Listing A.2: EnemyData JSON schema and example entries

Relic Data

```

1 {
2     "Relics": [
3         {
4             "Id": "burning_ember",
5             "Name": "Burning Ember",
6             "Tier": "Rare",           // Common | Uncommon | Rare | Boss
7             "TriggerType": "StartOfCombat",
8             "Actions": [
9                 { "Type": "ApplyEffect", "TargetType": "AllEnemies",
10                  "EffectId": "burn", "Value": 3 }
11             ]
12         },
13         {
14             "Id": "iron_will",
15             "Name": "Iron Will",
16             "Tier": "Common",
17             "TriggerType": "OnReceiveDamage",
18             "Condition": "HeroHpBelow30Pct",
19             "Actions": [

```

```

20         { "Type": "GainBlock", "TargetType": "Self", "Value": 6 }
21     ]
22 }
23 ]
24 }

```

Listing A.3: RelicData JSON schema and example entry

Algorithm Pseudocode

Single-Objective Genetic Algorithm

Algorithm 1 Single-Objective Genetic Algorithm for Game Balancing

Require: Population size $N = 100$, generations $G = 30$, simulations per genome $S = 200$

```

1:  $P \leftarrow \text{InitPopulation}(N)$  ▷ Each genome  $\sim \mathcal{U}(0.7, 1.3)^{500}$ 
2: for  $g = 1$  to  $G$  do
3:   EvaluateParallel( $P, S$ ) ▷ 8-core parallel eval.
4:    $P_{\text{elite}} \leftarrow \text{TopK}(P, 0.1N)$  ▷ Elitism: keep top 10%
5:    $P_{\text{offspring}} \leftarrow \emptyset$ 
6:   while  $|P_{\text{offspring}}| < N - |P_{\text{elite}}|$  do
7:      $p_1 \leftarrow \text{TournamentSelect}(P, k = 3)$ 
8:      $p_2 \leftarrow \text{TournamentSelect}(P, k = 3)$ 
9:      $c \leftarrow \text{UniformCrossover}(p_1, p_2, p_c = 0.80)$ 
10:     $c \leftarrow \text{GaussianMutate}(c, p_m = 0.02, \sigma = 0.10)$ 
11:     $P_{\text{offspring}} \leftarrow P_{\text{offspring}} \cup \{c\}$ 
12:   end while
13:    $P \leftarrow P_{\text{elite}} \cup P_{\text{offspring}}$ 
14: end for
15: return BestFitness( $P$ )

```

NSGA-II Multi-Objective Optimizer

Algorithm 2 NSGA-II for Multi-Objective Game Balancing

Require: Population size $N = 50$, generations $G = 30$

```

1:  $P \leftarrow \text{InitPopulation}(N)$ 
2:  $\text{Evaluate}(P, S = 80)$  ▷ 20 simulations  $\times$  4 agents per genome
3: for  $g = 1$  to  $G$  do
4:    $\mathcal{F} \leftarrow \text{FastNonDominatedSort}(P)$  ▷ Returns fronts  $F_1, F_2, \dots$ 
5:    $\text{ComputeCrowdingDistance}(\mathcal{F})$ 
6:    $Q \leftarrow \text{BinaryTournamentSelect}(P)$  ▷ By rank then crowding dist.
7:    $Q \leftarrow \text{SBXCrossover}(Q, \eta_c = 20, p_c = 0.90)$ 
8:    $Q \leftarrow \text{PolynomialMutate}(Q, \eta_m = 20, p_m = 0.05)$ 
9:    $R \leftarrow P \cup Q$  ▷ Combined parent + offspring
10:   $\text{Evaluate}(R \setminus P, S = 80)$  ▷ Evaluate new offspring, 20 sims  $\times$  4 agents
11:   $\mathcal{F}' \leftarrow \text{FastNonDominatedSort}(R)$ 
12:   $\text{ComputeCrowdingDistance}(\mathcal{F}')$ 
13:   $P \leftarrow \text{SelectBestN}(R, N)$  ▷ By rank, then crowding distance
14: end for
15: return  $F_1$  (first Pareto front of final  $P$ )
```

Observation Space Index Table

Table A.1: 152-dimensional observation vector layout. All values are normalized to $[0, 1]$ unless noted.

Index	Feature	Normalization	Notes
0	Hero HP ratio	$\div \text{max HP}$	1.0 = full HP
1	Hero block ratio	$\div 40$	Can exceed 1.0
2	Mana ratio	$\div \text{max mana}$	
3	Gold ratio	$\div 500$	
4-10	Hero status effect intensities (7 effects)	$\div 10$	Clipped at 1.0
11-80	Hand cards (10 slots $\times$ 7 features)	Various	Cost, star, type, #actions, playable, dmg, block

Continued on next page

Index	Feature	Normalization	Notes
81-85	Deck star-count histogram (stars 1-5)	$\div 20$	Count per star tier
86-109	Enemy state (4 slots $\times$ 6 features)	Various	HP, block, star, #effects, intent type, intent value
110-119	Equipped relic star ratings (10 slots)	$\div 5$	0 = empty slot
120-126	Game state one-hot (7 states)	Binary	PreRun, OnMap, InCombat, ...
127-140	Map next nodes (7 slots $\times$ 2 features)	Various	Room type ratio, star rating
141	Current floor ratio	$\div 16$	
142	Combat turn ratio	$\div 20$	
143	Draw pile size ratio	$\div 30$	
144	Total incoming damage ratio	$\div 60$	Sum of enemy attack intents
145	Affordable hand damage ratio	$\div 50$	Playable cards only
146	Combat turn ratio (alt)	$\div 10$	Duplicate for agent convenience
147	Attack/Skill ratio in deck	-	$n_{\text{atk}} / (n_{\text{atk}} + n_{\text{def}})$
148	Cards played this turn	$\div 5$	
149	Affordable block ratio	$\div 20$	Playable block cards only
150	Block coverage ratio	-	block $\div$ incoming dmg
151	Current floor ratio (2nd)	$\div 16$	Redundant; retained for compatibility

Reward Function Summary

Table A.2 summarizes the reward functions of all four RL agents. The deck quality reward (Section 3.5.3) is shared by all agents.

Table A.2: Complete reward function summary for all four RL agents.

Agent	Step Reward	Terminal Reward
Aggressive	-0.10 per step; HP-loss penalty $-0.10 \cdot \Delta\text{HP} \cdot m(\text{HP}\%)$ where $m = 3$ at $\text{HP} < 25\%$, $m = 2$ at $\text{HP} < 40\%$, $m = 1$ otherwise; $+5.0$ per enemy killed	$+500 + 15 \cdot (1 - t/t_{\max})$ on win (speed bonus); -200 on death
Defensive	-0.005 per step; $+0.30 \cdot \Delta\text{HP}_{\text{healed}}$; $-0.15 \cdot \min(\Delta\text{HP}_{\text{lost}} , 20)$; $+0.30 \cdot \Delta\text{Block}$; $+2.0$ per combat survived	$+500 + 80 \cdot (\text{HP}/\text{HP}_{\max})$ on win; -200 on death
Balanced	-0.01 per step; $+0.20 \cdot \Delta\text{HP}_{\text{healed}}$; $-0.20 \cdot \Delta\text{HP}_{\text{lost}} $; $+0.02 \cdot \Delta\text{gold}$	$+500 + 30 \cdot (\text{HP}/\text{HP}_{\max})^2$ on win; -200 on death
Adaptive	-0.01 per step; HP-scaled loss penalty $-(0.1 + 0.3(1 - \text{HP}\%)) \cdot \Delta\text{HP} $; $+0.2$ per block when $\text{HP} < 40\%$; $+25.0$ per floor; -0.05 idle stagnation; $+2.0$ per kill	$+500$ on win; -200 on death
Shared across all agents:		
Deck quality: $+8$ (4-5★ card selected), $+3$ (3★ card), 0 (1-2★ card)		

Code Listings

Python Package Structure

```
1 python_roguelike/
2 | - core/
3 |   | - ai/
4 |   |   | - heuristic_player_ai.py    # Deterministic heuristic agent
5 |   |   | - i_player_agent.py        # Interface for all agents
6 |   |   | - combat_manager.py        # Turn-by-turn combat logic
7 |   |   | - deck_manager.py          # Draw/discard pile management
8 |   |   | - active_effect.py         # Status effect runtime
9 |   |   | - game_controller.py       # Top-level game run
10 |   |   | orchestrator
11 |   |   | - game_run.py              # Mutable run state object
12 |   |   | - map/
13 |   |   |   | - map_generator.py     # Procedural map generation
14 |   |   |   | - map_graph.py        # DAG of room nodes
15 |   |   |   | - rooms/              # One handler per room type
16 |   |   |   | - data/               # Data classes and enums
17 |   |   |   |   | - enums.py        # GameState, RoomType, CardType,
18 |   |   |   |   | ...
19 |   |   |   |   | - card_data.py, enemy_data.py, relic_data.py, ...
20 |   |   |   |   | - pools/         # CardPool, EnemyPool, RelicPool
21 |   |   |   |   | , ...
22 |   |   |   | - env/
23 |   |   |   |   | - roguelike_env.py # Gymnasium wrapper (
24 |   |   |   |   |   RoguelikeEnv)
25 |   |   |   | - optimization/
26 |   |   |   |   | - hierarchical_genome.py    # 38-dimensional genome
27 |   |   |   |   | - hierarchical_applicator.py # Applies genome to game data
28 |   |   |   |   | - simulation_runner.py      # Batch simulation with PPO
29 |   |   |   |   | agents
30 |   |   |   |   | - fitness.py                # MultiObjectiveEvaluator
31 |   |   |   |   | - nsga2_problem.py          # pymoo Problem definition
32 |   |   |   |   | - run_optimization.py       # CLI entry point
33 |   |   |   | - training/
34 |   |   |   |   | - training_utils.py        # Phase constants and callbacks
35 |   |   |   |   | - data_loader.py           # Parses GameData.json
```



```
31 | - GameData.json                                # All game entities
```

Listing A.4: Python package structure

Game Controller Initialization

```
1 class GameController:
2     def __init__(self, card_pool, relic_pool, enemy_pool,
3                 effect_pool, event_pool, room_configs):
4         self._handlers = {
5             RoomType.Monster: CombatRoomHandler(),
6             RoomType.Elite:    CombatRoomHandler(),
7             RoomType.Boss:     BossRoomHandler(),
8             RoomType.Event:    EventRoomHandler(),
9             RoomType.Shop:     ShopRoomHandler(),
10            RoomType.Rest:     RestRoomHandler(),
11        }
12        self.current_run: Optional[GameRun] = None
13
14    def start_new_run(self, seed: int, hero_data: HeroData):
15        self.current_run = GameRun(seed, hero_data, ...)
16
17    def choose_map_node(self, node_id: int) -> bool:
18        room = self.current_run.the_map.get_current_room()
19        handler = self._handlers.get(room.type)
20        if handler:
21            handler.execute(self.current_run, room)
22        return True
```

Listing A.5: GameController initialization (simplified)

Example Card Definition

```
1 {
2     "Id": "flame_strike",
3     "Name": "Flame Strike",
4     "Type": "Attack",
5     "ManaCost": 2,
6     "StarRating": 3,
```

```

7  "Actions": [
8    { "Type": "DealDamage", "Target": "AllEnemies", "Value": 8 },
9    { "Type": "ApplyStatusEffect", "Target": "AllEnemies",
10      "EffectId": "burn", "Value": 3 }
11  ]
12 }

```

Listing A.6: Example card definition in GameData.json

Heuristic AI Decision Logic

```

1  class HeuristicPlayerAI(IPlayerAgent):
2
3      def get_combat_decision(self, run) -> CombatDecision:
4          hero      = run.the_hero
5          enemies   = [e for e in run.current_combat.enemies
6                      if e.current_health > 0]
7          incoming  = self._calc_incoming_damage(run, enemies)
8          needed_block = max(0, incoming - hero.block)
9
10         # Priority 1: Survival
11         if needed_block > 0:
12             move = self._find_best_defensive_move(hero, needed_block)
13             if move: return move
14
15         # Priority 2: Lethal
16         move = self._find_lethal_move(hero, enemies)
17         if move: return move
18
19         # Priority 3: Best general offensive move
20         return self._get_best_general_move(hero, enemies)
21
22     def choose_map_node(self, run) -> int:
23         hp_pct = run.the_hero.current_health / run.the_hero.
24                 max_health
25         return max(run.the_map.get_possible_next_nodes(),
26                   key=lambda n: self._evaluate_room(run, n, hp_pct))
27                 .id

```

Listing A.7: Combat decision logic of HeuristicPlayerAI (simplified)

Core Gymnasium Environment Methods

```
1 OBS_DIM = 152    # see Table 3.2
2 ACT_DIM = 67     # see Section 3.3.2
3
4 class RoguelikeEnv(gym.Env):
5     def reset(self, *, seed=None, options=None):
6         if self._domain_randomization:
7             self._apply_domain_randomization()
8         self._controller = GameController(
9             self._card_pool, self._relic_pool, ...)
10        self._controller.start_new_run(seed, self._hero_data)
11        return self._get_obs(), {}
12
13    def step(self, action: int):
14        run = self._controller.current_run
15        self._apply_action(action, run)
16        obs = self._get_obs()
17        reward = self._compute_reward(run)
18        done = (run.current_state == GameState.GameOver)
19        info = {"win": run.is_victory, "floor": run.current_floor}
20        return obs, reward, done, False, info
21
22    def action_masks(self) -> np.ndarray:
23        mask = np.zeros(ACT_DIM, dtype=bool)
24        for a in self._get_valid_actions():
25            mask[a] = True
26        return mask
```

Listing A.8: Core Gymnasium environment methods (simplified)

Curriculum Domain Randomization

```
1 def _apply_domain_randomization(self) -> None:
2     rng = self.np_random
```

```

3     if self._dr_curriculum:
4         # Ramp noise linearly from 0 to dr_noise over
5         # dr_warmup_episodes
6         progress = min(self._episode_count /
7                        max(self._dr_warmup_episodes, 1), 1.0)
8         noise = self._dr_noise * progress
9     else:
10        noise = self._dr_noise
11    lo, hi = 1.0 - noise, 1.0 + noise
12
13    for cid, card in self._card_pool.cards_by_id.items():
14        orig_mana, orig_vals = self._orig_card_values[cid]
15        card.mana_cost = max(1, round(orig_mana * rng.uniform(lo, hi)
16                                   ))
17        for i, action in enumerate(card.actions):
18            if orig_vals[i] > 0:
19                action.value = max(1, round(orig_vals[i] *
20                                           rng.uniform(lo, hi)))
21
22    for eid, enemy in self._enemy_pool.enemies_by_id.items():
23        enemy.starting_health = max(
24            1, round(self._orig_enemy_hp[eid] * rng.uniform(lo, hi)))
25
26    self._hero_data.starting_health = max(
27        1, round(self._orig_hero_hp * rng.uniform(lo, hi)))

```

Listing A.9: Curriculum Domain Randomization in reset()

Three-Phase Training Loop

```

1 from sb3_contrib import MaskablePP0
2 from python_roguelike.training.training_utils import (
3     DRCurriculumCallback, MaskableEvalCB, cosine_lr_fn,
4     PHASE1_STEPS, PHASE2_STEPS, PHASE3_STEPS,
5     P1_LR_START, P1_LR_END, P1_N_EPOCHS, P1_ENT_COEF, P1_CLIP_RANGE,
6     P2_LR_START, P2_LR_END, P2_N_EPOCHS, P2_ENT_COEF, P2_CLIP_RANGE,
7     P3_LR_START, P3_LR_END, P3_N_EPOCHS, P3_ENT_COEF, P3_CLIP_RANGE,
8     P3_DR_TARGET, P3_DR_RAMP_STEPS
9 )

```

```

10
11 # Phase 1 - easy maps (early_win_floor=8, no DR)
12 env_p1 = make_vec_env(lambda: RoguelikeEnv(early_win_floor=8), n_envs
    =4)
13 model = MaskablePPO(
14     "MlpPolicy", env_p1,
15     learning_rate=cosine_lr_fn(P1_LR_START, P1_LR_END),
16     n_steps=2048, batch_size=256, n_epochs=P1_N_EPOCHS,
17     ent_coef=P1_ENT_COEF, clip_range=P1_CLIP_RANGE,
18     policy_kwargs=dict(net_arch=[512, 512])
19 )
20 model.learn(PHASE1_STEPS, callback=MaskableEvalCB(eval_env, 20))
21 model.save("phase1_ckpt")
22
23 # Phase 2 - full maps, no DR
24 env_p2 = make_vec_env(lambda: RoguelikeEnv(), n_envs=4)
25 model = MaskablePPO.load("phase1_ckpt", env=env_p2)
26 model.learning_rate = cosine_lr_fn(P2_LR_START, P2_LR_END)
27 model.n_epochs = P2_N_EPOCHS
28 model.learn(PHASE2_STEPS, callback=MaskableEvalCB(eval_env, 20))
29 model.save("phase2_ckpt")
30
31 # Phase 3 - DR enabled with curriculum ramp
32 env_p3 = make_vec_env(
33     lambda: RoguelikeEnv(domain_randomization=True,
34                           dr_noise=P3_DR_TARGET,
35                           dr_warmup_episodes=P3_DR_RAMP_STEPS // 4),
36     n_envs=4)
37 model = MaskablePPO.load("phase2_ckpt", env=env_p3)
38 model.learning_rate = cosine_lr_fn(P3_LR_START, P3_LR_END)
39 model.n_epochs      = P3_N_EPOCHS
40 model.ent_coef      = P3_ENT_COEF
41 model.clip_range    = P3_CLIP_RANGE
42 model.learn(PHASE3_STEPS, callback=[DRCurriculumCallback(env_p3),
43                                     MaskableEvalCB(eval_env, 20)])
44 model.save("final_ckpt")

```

Listing A.10: Three-phase training loop (simplified)

Batch Genome Evaluation

```
1 class SimulationRunner:
2     def run_batch(self, genome: HierarchicalGenome,
3                   n_runs: int = 30,
4                   base_seed: int = 42) -> List[SimulationStats]:
5         # Clone base data pools and apply genome
6         card_pool = copy.deepcopy(self._base_card_pool)
7         enemy_pool = copy.deepcopy(self._base_enemy_pool)
8         hero      = copy.deepcopy(self._base_hero)
9         hierarchical_applicator.apply(genome, card_pool, enemy_pool,
10                                     hero)
11
12         results = []
13         for i in range(n_runs):
14             agent = self.agents[i % len(self.agents)] # round-robin
15             stats = self._run_single(agent, card_pool, enemy_pool,
16                                     hero, seed=base_seed + i)
17             results.append(stats)
18         return results
```

Listing A.11: Batch genome evaluation with PPO agents

pymoo Problem Definition

```
1 from pymoo.algorithms.moo.nsga2 import NSGA2
2 from pymoo.operators.crossover.sbx import SBX
3 from pymoo.operators.mutation.pm import PM
4 from pymoo.optimize import minimize
5
6 problem = RoguelikeBalanceProblem(
7     runner=runner,
8     evaluator=MultiObjectiveEvaluator(),
9     runs_per_genome=20,
10 )
11
12 algorithm = NSGA2(
13     pop_size=50,
14     crossover=SBX(eta=20, prob=0.9),
```

```

15     mutation=PM(eta=20),
16     eliminate_duplicates=True,
17 )
18
19 result = minimize(
20     problem,
21     algorithm,
22     termination=get_termination("n_gen", 30),
23     seed=42,
24     verbose=True,
25 )
26 # result.X: Pareto-front parameter vectors
27 # result.F: corresponding (negated) objective values

```

Listing A.12: pymoo Problem definition and optimization launch

BIBLIOGRAPHY

- [1] Mega Crit Games, “Slay the spire,” PC game. Available at: <https://store.steampowered.com/app/646570>, 2019.
- [2] J. Schell, *The Art of Game Design: A Book of Lenses*, 3rd ed. Boca Raton, FL: CRC Press, 2019.
- [3] J. Dormans, “Level design as model transformation: A strategy for automated content generation,” in *Proceedings of the Second International Workshop on Procedural Content Generation in Games*, 2011.
- [4] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [7] S. Huang and S. Ontañón, “A closer look at invalid action masking in policy gradient algorithms,” *arXiv preprint arXiv:2006.14171*, 2020.
- [8] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [9] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.
- [10] M. J. Nelson and M. Mateas, “Towards automated game design,” in *Proceedings of the Congress of the Italian Association for Artificial Intelligence (AI*IA 2007)*, ser. Lecture Notes in Computer Science, vol. 4733. Springer, 2007, pp. 626–637.

- [11] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Cambridge, MA: MIT Press, 1992.
- [12] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [13] C. A. Coello Coello, G. B. Lamont, and D. A. Van Veldhuizen, *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2nd ed. New York, NY: Springer, 2007.
- [14] O. Vinyals, I. Babuschkin, W. M. Czarnecki *et al.*, “Grandmaster level in StarCraft II using multi-agent reinforcement learning,” *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.
- [15] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [16] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, “Search-based procedural content generation: A taxonomy and survey,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011.
- [17] M. Preuss, A. Liapis, and J. Togelius, “Searching for good and diverse game levels,” in *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, 2014, pp. 1–8.
- [18] J. Blank and K. Deb, “pymoo: Multi-Objective optimization in Python,” *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.